

capstone

Owen McGinnis

2025-10-06

```
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':  
##  
##   filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
library(stringr)  
library(data.table)
```

```
##  
## Attaching package: 'data.table'
```

```
## The following objects are masked from 'package:dplyr':  
##  
##   between, first, last
```

```
library(tidyverse)
```

```
## Warning: package 'tidyr' was built under R version 4.5.1
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —  
## ✓ forcats   1.0.0   ✓ readr      2.1.4  
## ✓ ggplot2   3.4.4   ✓ tibble     3.2.1  
## ✓ lubridate 1.9.3   ✓ tidyr      1.3.2  
## ✓ purrr     1.0.2
```

```
## — Conflicts — tidyverse_conflicts() —  
## * data.table::between() masks dplyr::between()  
## * dplyr::filter()       masks stats::filter()  
## * data.table::first()   masks dplyr::first()  
## * lubridate::hour()     masks data.table::hour()  
## * lubridate::isoweek()  masks data.table::isoweek()
```

```
## * lubridate::isoyear() masks data.table::isoyear()
## * dplyr::lag() masks stats::lag()
## * data.table::last() masks dplyr::last()
## * lubridate::mday() masks data.table::mday()
## * lubridate::minute() masks data.table::minute()
## * lubridate::month() masks data.table::month()
## * lubridate::quarter() masks data.table::quarter()
## * lubridate::second() masks data.table::second()
## * purrr::transpose() masks data.table::transpose()
## * lubridate::wday() masks data.table::wday()
## * lubridate::week() masks data.table::week()
## * lubridate::yday() masks data.table::yday()
## * lubridate::year() masks data.table::year()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(stats)
library(ggthemes)
library(ggmosaic)
library(lubridate)
library(hms)
```

```
##
## Attaching package: 'hms'
##
## The following object is masked from 'package:lubridate':
##
##     hms
```

```
library(plotly)
```

```
##
## Attaching package: 'plotly'
##
## The following object is masked from 'package:ggplot2':
##
##     last_plot
##
## The following object is masked from 'package:stats':
##
##     filter
##
## The following object is masked from 'package:graphics':
##
##     layout
```

```
library(rmdformats)
```

Cleaning Chunk 1

```
##cleaning chunk
schedule_data = read.csv("/Users/omcginnis/Downloads/Fall 2025_Course Schedule Master_F
or STA 382_082225(1).xlsx - WFU - OUR - Course Schedule Mas.csv") ## reading in data
colnames(schedule_data) = gsub("\\.", "_", colnames(schedule_data)) ##not necessary, re
placing all commas with underscores
final_wake_space = schedule_data |>
  filter(Campus == "Wake Downtown Campus (IQ)" | Campus == "Reynolda Campus") |> ## sel
ecting only on-campus classes(getting rid of abroad)
  filter(Section_Status == "Open" | Section_Status == "Waitlist" | Section_Status == "C
losed") |> ## getting rid of cancelled classes
  filter(Department != "School of Business") |>
  filter(Department != "School of Divinity") |> ## getting rid of Business School and D
ivinity School
  filter(Academic_Level == "Undergraduate") |> ## selecting only undergrad courses
  filter(Instructional_Format != "Independent Study" & Instructional_Format != "Researc
h") |> ## these types had no rooms assigned, so not relevant to our question
  filter(Section_Capacity != 0) |>
  filter(Enrollment_Count != 0) |>
  select(c("Academic_Period", "Department", "Campus", "Course_Number", "Section_Num
ber", "Section", "Section_Status", "Enrollment_Count", "Section_Capacity", "Total_Sectio
n_Seat_s_Available", "Wait_List_Capacity", "Waitlist_Count", "Meeting_Patterns", "Meeti
ng_Day_Patterns", "Locations__Building_Room_", "Delivery_Mode", "Instructional_Forma
t", "Course_Section_Overlaps")) ## selecting various columns
overlaps = c()
for(i in 1:nrow(final_wake_space)){

  if(final_wake_space[i, "Course_Section_Overlaps"] != ""){
    overlaps <- c( overlaps, which(final_wake_space$Section == final_wake_space[i, "Cour
se_Section_Overlaps"]))
  }
}

## essentially, if a section has an overlap, this code guarantees there is only one lis
ting for that section
```

Cleaning Chunk 2

```
##Correcting all sections with double room assignments or double meeting patterns
reynolda_space = final_wake_space |>
  filter(Campus == "Reynolda Campus")
reynolda_space$ID <- 1:nrow(reynolda_space) ## giving each row a number to refer to late
r
doubles = reynolda_space |>
  filter(grepl("\n\n", Locations__Building_Room_) == TRUE) ## checking which rows are p
roblem rows(have two locations listed)
doubles_list = c()

for(i in 1:nrow(doubles)){
  doubles_list[i] = doubles[i, "ID"] ##holding row numbers for iteration
}
list = c()
```

```

for(i in 1:nrow(doubles)){
  new = reynolda_space[c(doubles_list[i],doubles_list[i]),] ## generating a new data frame
  to hold the duplicated rows
  holder <-unlist(strsplit(reynolda_space[doubles_list[i],"Meeting_Patterns"], split="\n
\n")) ## splitting the string on the R-generated character for a space

  new[1,"Meeting_Patterns"] <-holder[1]
  new[2,"Meeting_Patterns"] <-holder[2] ## row 1 gets what is before the "\n\n" and row 2
  gets what is after
  ##

  holder <-unlist(strsplit(reynolda_space[doubles_list[i],"Meeting_Day_Patterns"],split
  ="\n\n"))

  new[1,"Meeting_Day_Patterns"] <-holder[1]
  new[2,"Meeting_Day_Patterns"] <-holder[2] ## same as above

  holder= unlist(strsplit(as.character(reynolda_space[doubles_list[i],"Locations__Buildin
  g_Room_"]),split="\n\n"))
  new[1,"Locations__Building_Room_"] <-holder[1]
  new[2,"Locations__Building_Room_"] <-holder[2]

  list[[i]] = new
  list_r_2 = rbindlist(list)## all the corrected rows combined into one set
} ## for each section with two different rooms listed, there also existed two different
meeting patterns, hence why we changed all 3 variables

filtered = reynolda_space[!grepl("\n\n", reynolda_space$Locations__Building_Room_), ] #
# removing all rows that had a double room(to allow us to reinsert the corrected two ro
ws in their place)
temp_space = rbind(filtered, list_r_2) ## placing the corrected rows back in the data s
et
temp_space$ID <-1:nrow(temp_space) ## now, from the resulting set, there are still some
sections with two meeting patterns, but not two locations
## the following is the exact same logic as before, just for meeting patterns
doubts = temp_space |>
  filter(grepl("\n\n", Meeting_Day_Patterns))
doubts_list = c()
for(i in 1:nrow(doubts)){
  doubts_list[i] = doubts[i,"ID"] ##holding row numbers for iteration
}
list2 = c()
for(i in 1:nrow(doubts)){
  new = temp_space[c(doubts_list[i],doubts_list[i]),]
  holder <-unlist(strsplit(temp_space[doubts_list[i],"Meeting_Patterns"], split="\n\n"))

  new[1,"Meeting_Patterns"] <-holder[1]
  new[2,"Meeting_Patterns"] <-holder[2]

  holder <-unlist(strsplit(temp_space[doubts_list[i],"Meeting_Day_Patterns"],split="\n
\n"))

  new[1,"Meeting_Day_Patterns"] <-holder[1]
  new[2,"Meeting_Day_Patterns"] <-holder[2]

```

```
list2[[i]] = new
list_r = rbindlist(list2)
}
filtered2 = temp_space[!grepl("\n\n", temp_space$Meeting_Day_Patterns), ]
reynolda_campus = rbind(filtered2, list_r) ## this is the final data set
```

Sanity Check

```
reynolda_campus |>
  group_by(Locations__Building_Room_) |>
  summarise(count = n())
```

```
## # A tibble: 195 × 2
##   Locations__Building_Room_ count
##   <chr>                  <int>
## 1 Angelou Residence Hall 104      2
## 2 Carswell Hall 005             10
## 3 Carswell Hall 014              5
## 4 Carswell Hall 016              5
## 5 Carswell Hall 018              6
## 6 Carswell Hall 019             11
## 7 Carswell Hall 027A             3
## 8 Carswell Hall 101              7
## 9 Carswell Hall 102             10
## 10 Carswell Hall 107             8
## # i 185 more rows
```

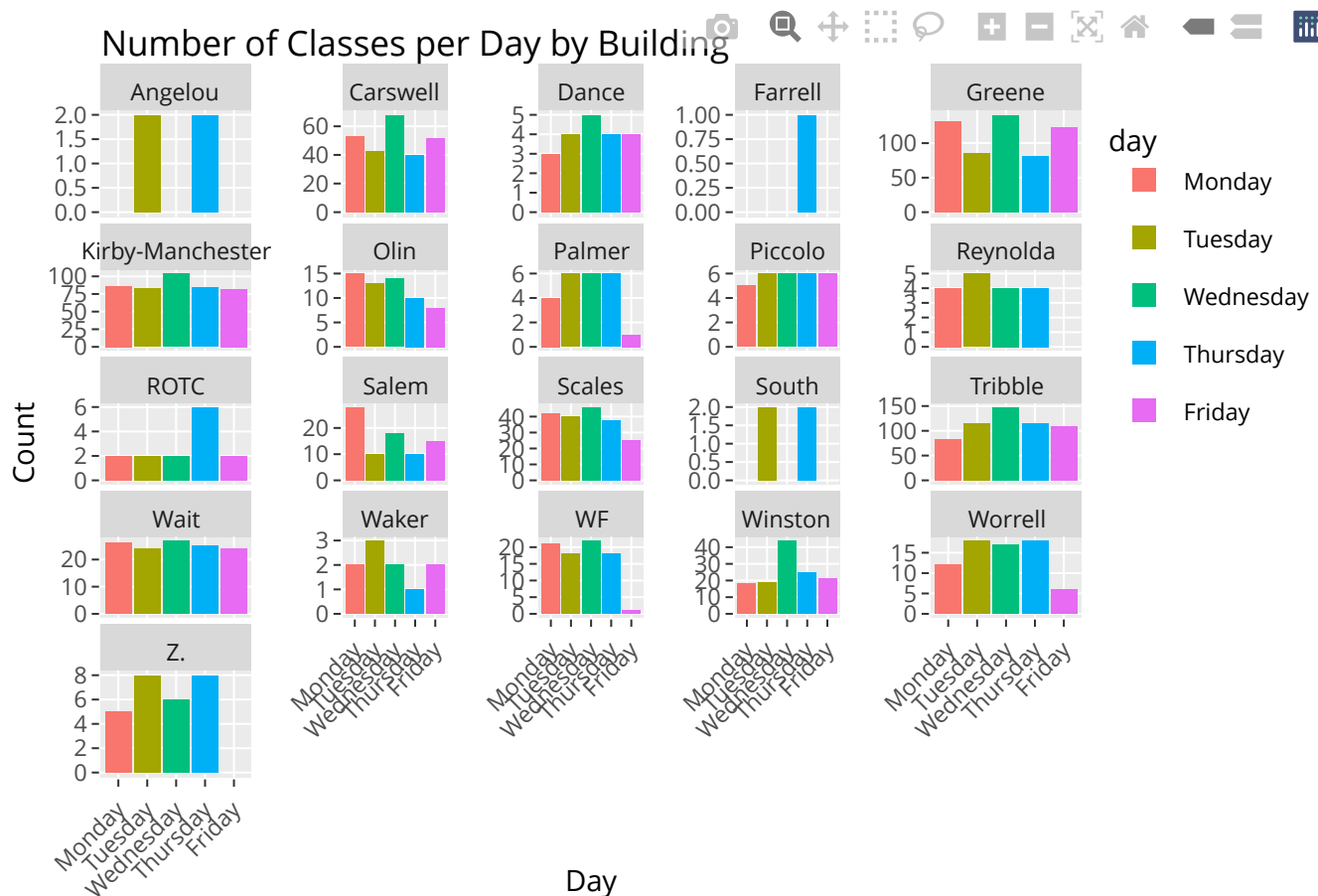
Crunch Time Investigation

```
by_building = reynolda_campus |>
  mutate(building = str_extract(Locations__Building_Room_, "^[A-Za-z]+")) |>
  group_by(building, Meeting_Patterns) |>
  mutate(days = str_extract(Meeting_Patterns, "^[^|]+"), is_Monday = str_detect(days,
" M"), is_Tuesday = str_detect(days, " T"), is_Wednesday = str_detect(days, " W"), is_Thur
sday = str_detect(days, " R"), is_Friday = str_detect(days, " F"))
  # summarise(count = n(), .groups = "drop")
by_building |>
  filter(building == "Farrell")
```

```
## # A tibble: 1 × 26
## # Groups:   building, Meeting_Patterns [1]
##   Academic_Period Department Campus      Course_Number Section_Number Section
##   <chr>          <chr>      <chr>      <chr>          <chr>      <chr>
## 1 2025 Fall Term Sociology Reynolda Camp... 365          A          SOC 36...
## # i 20 more variables: Section_Status <chr>, Enrollment_Count <int>,
## #   Section_Capacity <int>, Total_Section_Seat_s_Available <int>,
## #   Wait_List_Capacity <int>, Waitlist_Count <int>, Meeting_Patterns <chr>,
## #   Meeting_Day_Patterns <chr>, Locations__Building_Room_ <chr>,
## #   Delivery_Mode <chr>, Instructional_Format <chr>,
```

```
## # Course_Section_Overlaps <chr>, ID <int>, building <chr>, days <chr>,  
## # is_Monday <lgl>, is_Tuesday <lgl>, is_Wednesday <lgl>, is_Thursday <lgl>, ...
```

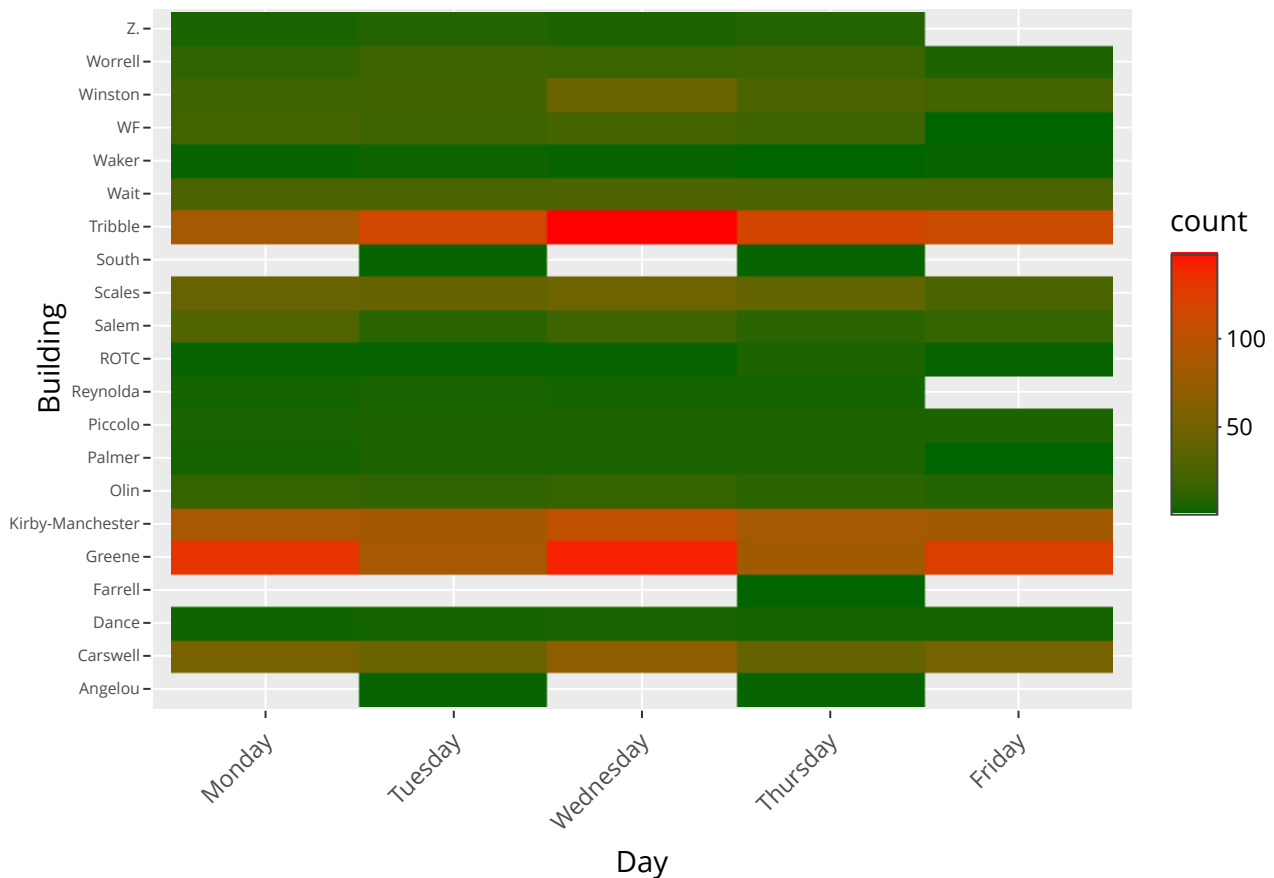
```
meetings_long = by_building |>  
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>  
  filter(has_class) |>  
  mutate(day = str_remove(day, "is_"))  
  
day_counts_by_building = meetings_long |>  
  mutate(building = word(Locations__Building_Room_,1)) |> ##need to read until the first  
  t space(just to adjust Kirby-Manchester)  
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday"))) |>  
  group_by(building, day) |>  
  summarise(count = n(), .groups = "drop")  
  
plotly::ggplotly(ggplot(day_counts_by_building, aes(x = day, y = count, fill = day)) +  
  geom_bar(stat = "identity") + facet_wrap(~ building, scales = "free_y") + labs(title =  
  "Number of Classes per Day by Building", x = "Day", y = "Count") + theme(axis.text.x =  
  element_text(angle = 45, hjust = 1)) )
```



```
plotly::ggplotly(ggplot(day_counts_by_building, aes(x = day, y = building, fill = count)) +  
  geom_tile(color = "white") + scale_fill_gradient(low = "darkgreen", high = "red") +  
  labs(title = "Number of Classes per Day by Building", x = "Day", y = "Building")
```

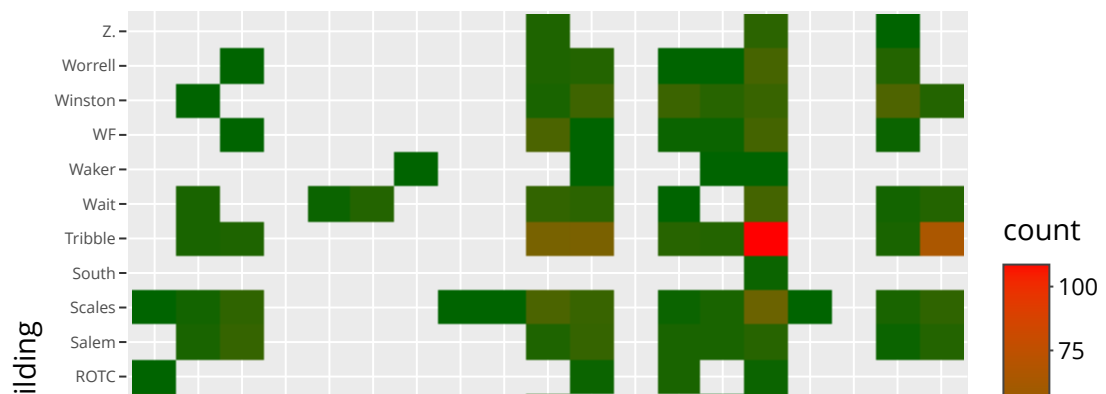
```
+ theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))
```

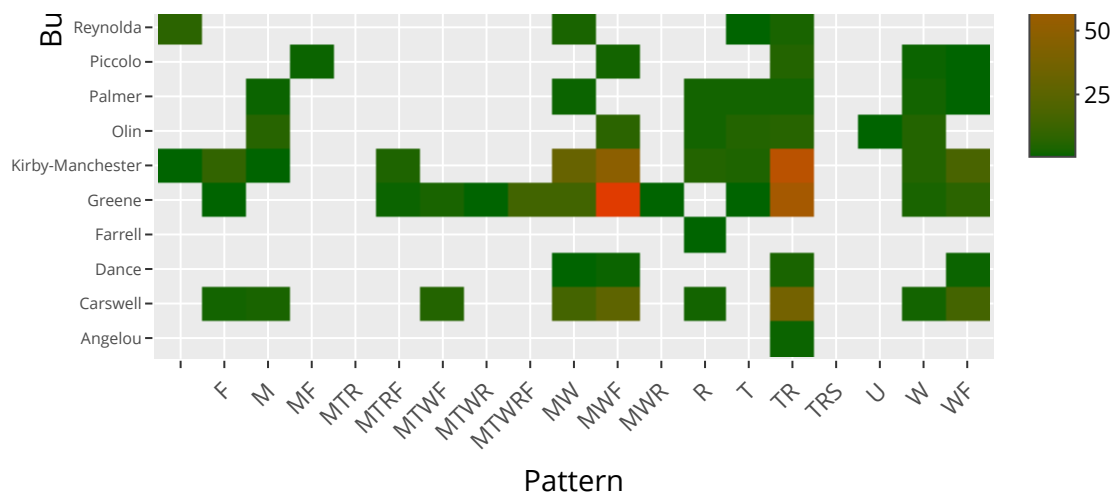
Number of Classes per Day by Building



```
pattern_counts_by_building = reynolda_campus |>
  mutate(building = word(Locations__Building_Room_,1)) |>
  group_by(building, Meeting_Day_Patterns) |>
  summarise(count = n(), .groups = "drop")
plotly::ggplotly(ggplot(pattern_counts_by_building, aes(x = Meeting_Day_Patterns, y = building, fill = count)) + geom_tile(color = "white") + scale_fill_gradient(low = "dark green", high = "red") + labs(title = "Number of Classes per Pattern by Building", x = "Pattern", y = "Building") + theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))
```

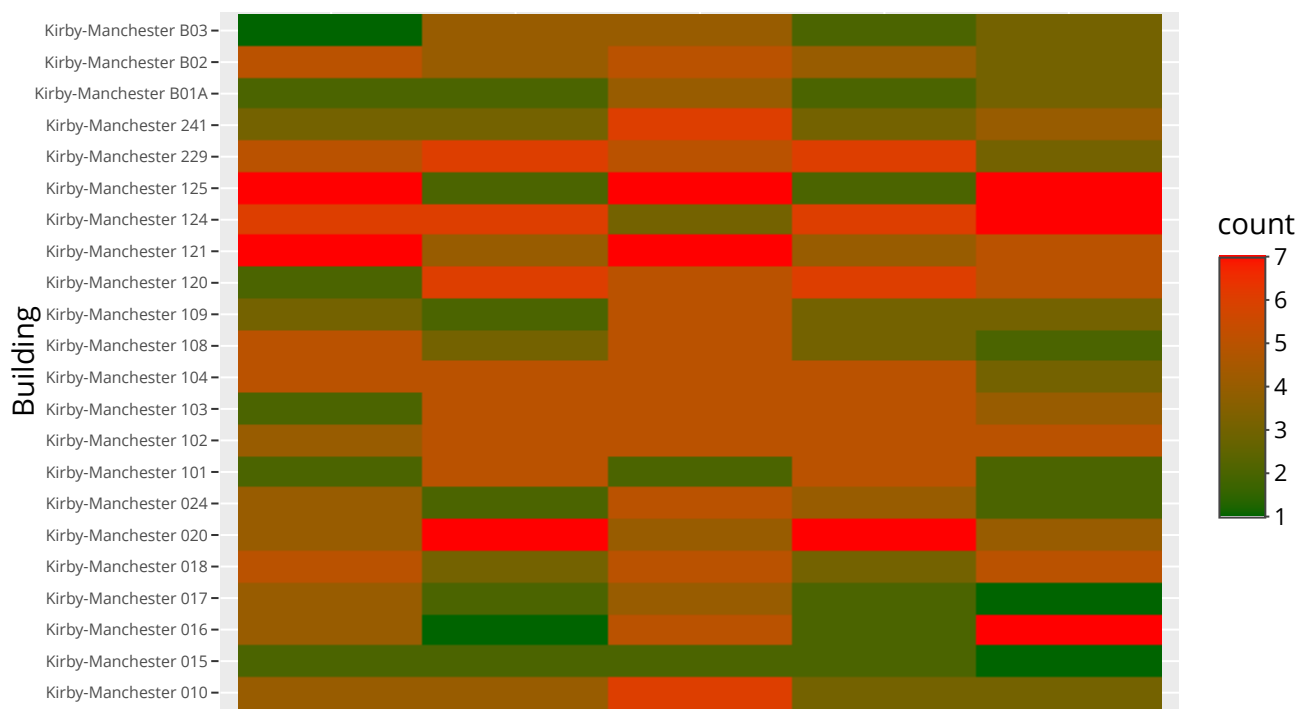
Number of Classes per Pattern by Building





```
## save for later, trying to create a heatmap for individual rooms(could even go by building and do all the room options by building), could also do the most common buildings (Kirby-Manchester, Greene, Carswell, Tribble, Winston, Salem, Olin, Scales, Wait Chapel, Worrell)
day_counts_by_room = meetings_long |>
  mutate(building = str_extract(Locations__Building_Room_, "^[A-Za-z]+")) |>
  filter(building == "Kirby") |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday"))) |>
  group_by(Locations__Building_Room_, day) |>
  summarise(count = n(), .groups = "drop")
plotly::ggplotly(ggplot(day_counts_by_room, aes(x = day, y = Locations__Building_Room_, fill = count)) + geom_tile(color = "white") + scale_fill_gradient(low = "darkgreen", high = "red") + labs(title = "Number of Classes per Day by Room for Kirby-Manchester", x = "Day", y = "Building") + theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))
```

Number of Classes per Day by Room for Kirby-Manchester



Monday

Tuesday

Wednesday

Thursday

Friday

Day

```

daypattern_counts_by_room = reynolda_campus |>
  mutate(building = str_extract(Locations__Building_Room_, "^[A-Za-z]+")) |>
  filter(building == "Kirby") |>
  group_by(Locations__Building_Room_, Meeting_Day_Patterns) |>
  summarise(count = n(), .groups = "drop")

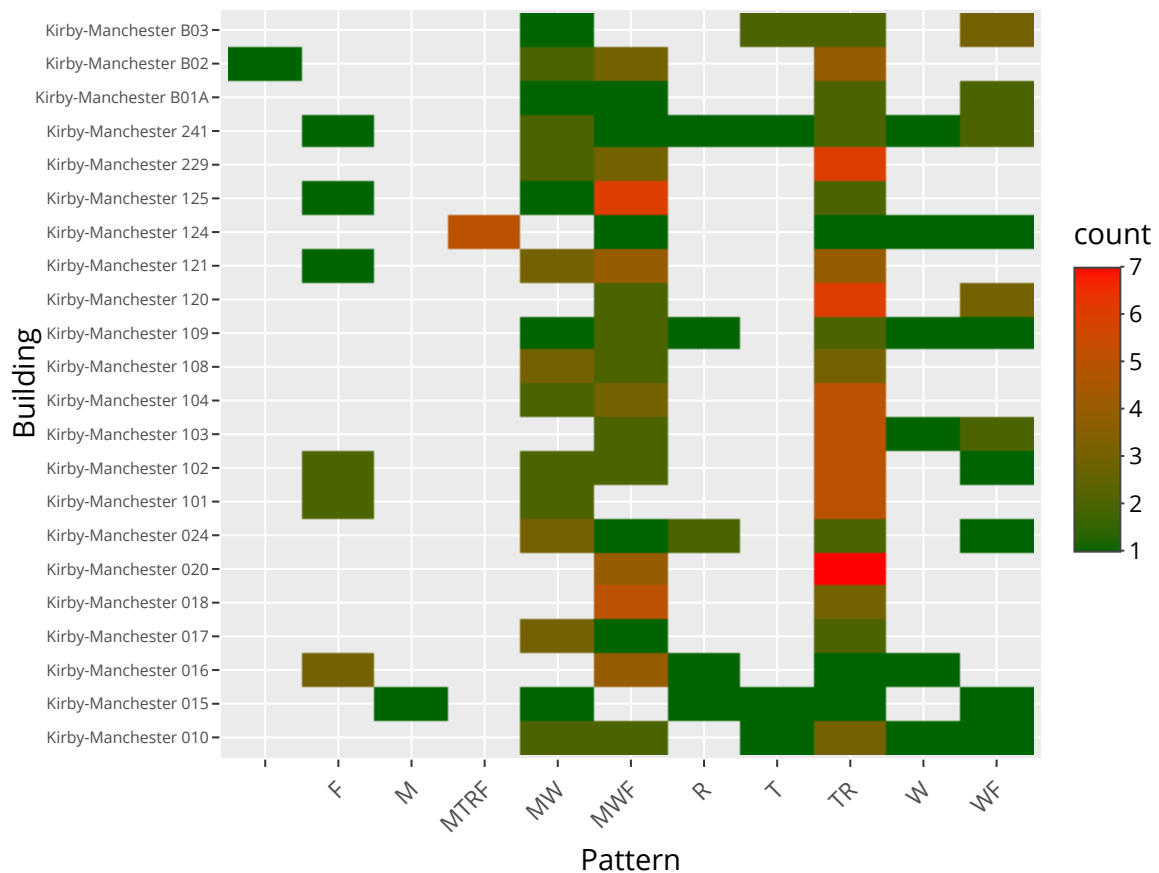
```

```

plotly::ggplotly(ggplot(daypattern_counts_by_room, aes(x = Meeting_Day_Patterns, y = Locations__Building_Room_, fill = count)) +
  geom_tile(color = "white") + scale_fill_gradient(low = "darkgreen", high = "red") +
  labs(title = "Number of Classes per Pattern by Room for Kirby-Manchester", x = "Pattern", y = "Building") +
  theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))

```

Number of Classes per Pattern by Room for Kirby-Manchester



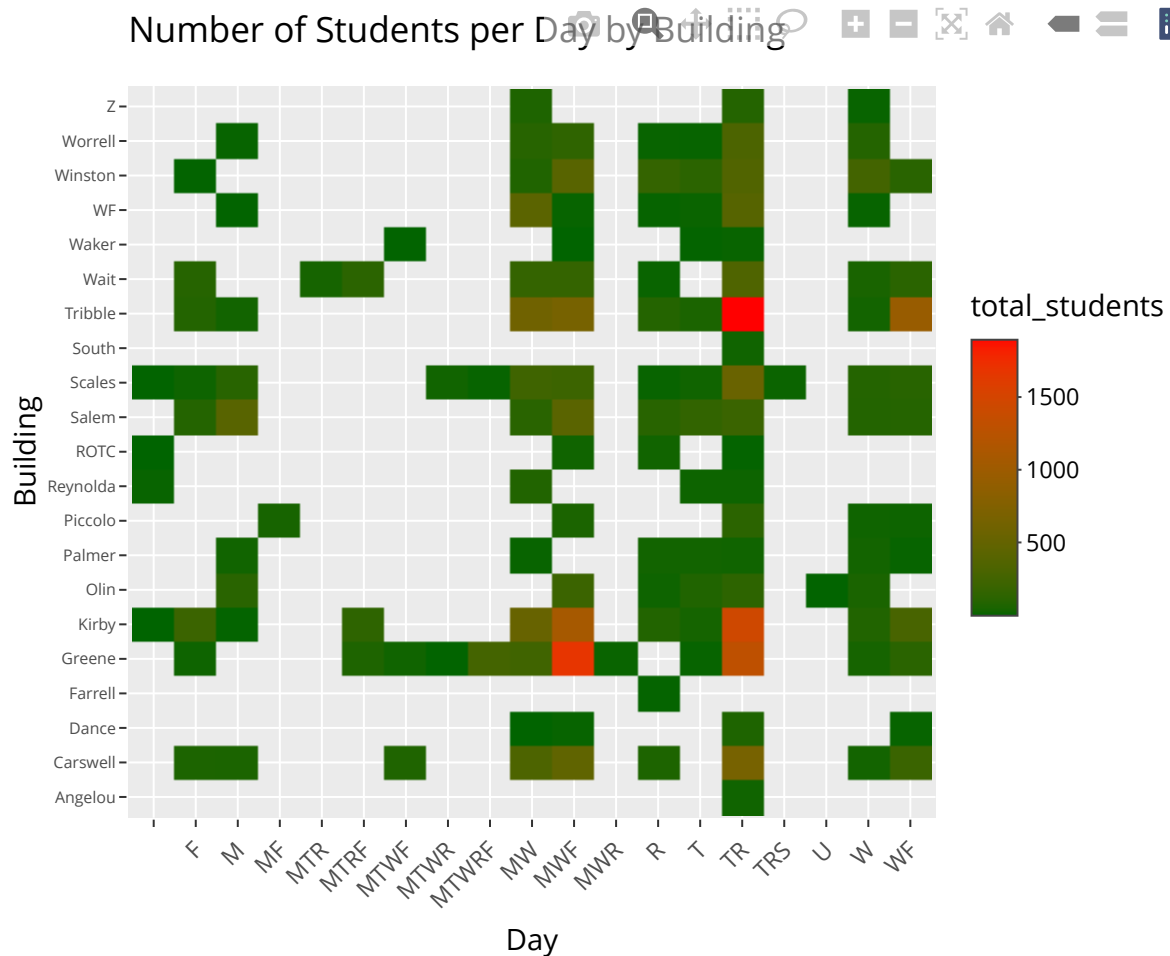
```

pattern_enrollment_by_building = reynolda_campus |>
  mutate(building = str_extract(Locations__Building_Room_, "^[A-Za-z]+")) |>
  group_by(building, Meeting_Day_Patterns) |>
  summarise(total_students = sum(Enrollment_Count))

```

`summarise()` has grouped output by 'building'. You can override using the
`.groups` argument.

```
plotly::ggplotly(ggplot(pattern_enrollment_by_building, aes(x = Meeting_Day_Patterns, y = building, fill = total_students)) + geom_tile(color = "white") + scale_fill_gradient(low = "darkgreen", high = "red") + labs(title = "Number of Students per Day by Building", x = "Day", y = "Building") + theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))
```

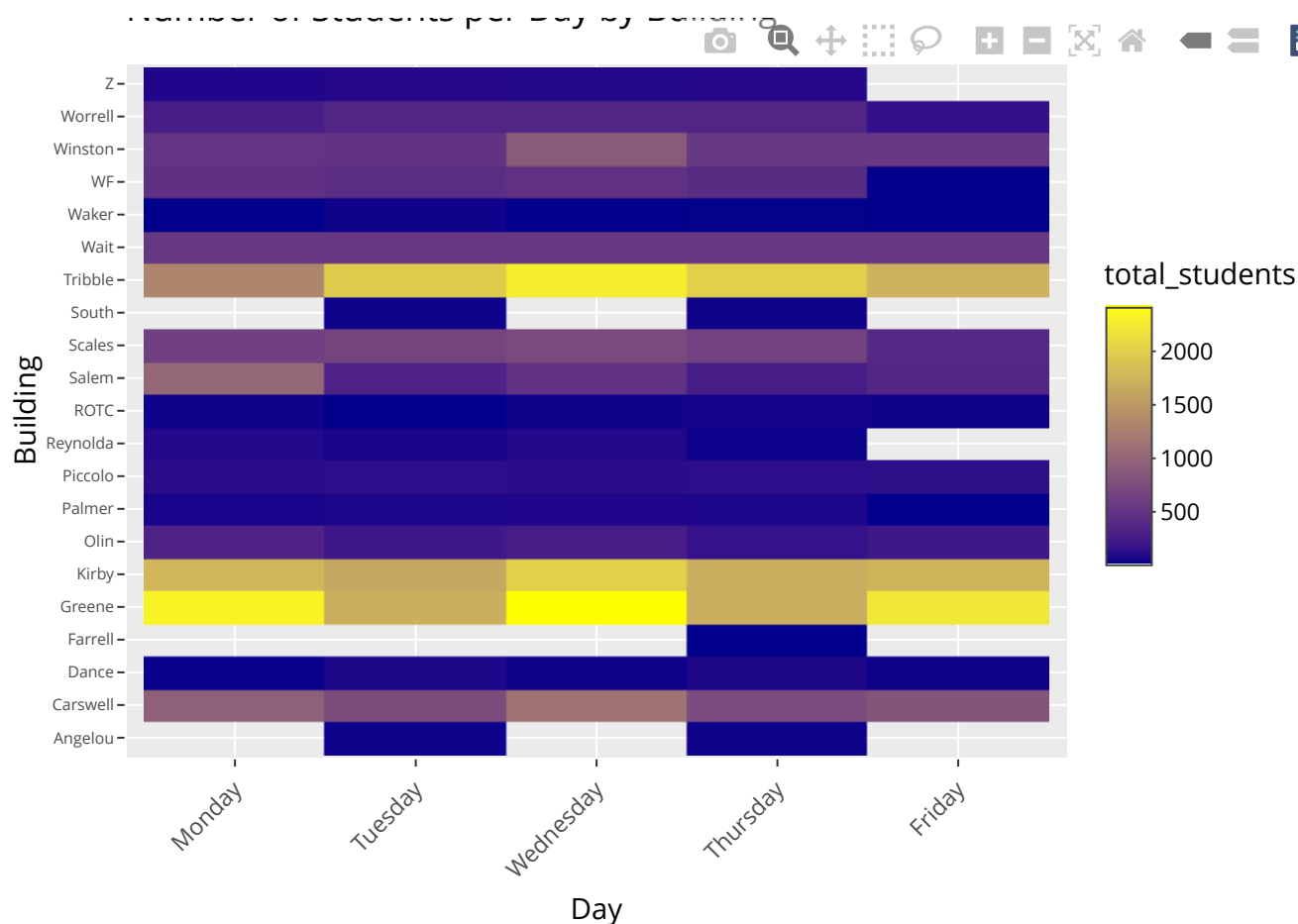


```
day_enrollment_by_building = meetings_long |>
  mutate(building = str_extract(Locations__Building_Room_, "[A-Za-z]+")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday"))) |>
  group_by(building, day) |>
  summarise(total_students = sum(Enrollment_Count))
```

`summarise()` has grouped output by 'building'. You can override using the
`.groups` argument.

```
plotly::ggplotly(ggplot(day_enrollment_by_building, aes(x = day, y = building, fill = total_students)) + geom_tile(color = "white") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs(title = "Number of Students per Day by Building", x = "Day", y = "Building") + theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))
```

Number of Students per Day by Building



```
peak_space = reynolda_campus |>
mutate(time_str = str_extract(Meeting_Patterns, regex("(?<=\\|)[^-]+")), time = as_hms
(parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:
30:00") ~ "pre_peak", time <= as_hms("13:59:59") ~ "peak", time <= as_hms("14:00:00") ~
"two_oclock", time >= as_hms("15:00:00") ~ "post_peak", TRUE ~ NA_character_)
)
post_peak = peak_space |>
  filter(time_block == "post_peak")
peak = peak_space |>
  filter(time_block == "peak")
pre_peak = peak_space |>
  filter(time_block == "pre_peak")
two_oclock = peak_space |>
  filter(time_block == "two_oclock")
```

Pie Chart

```
peak_numbers = peak_space |>
  mutate(days = str_extract(Meeting_Patterns, "^^[^|]+"), is_Monday = str_detect(days,
"M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thur
sday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
  filter(has_class) |>
  mutate(day = str_remove(day, "is_")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Fr
```

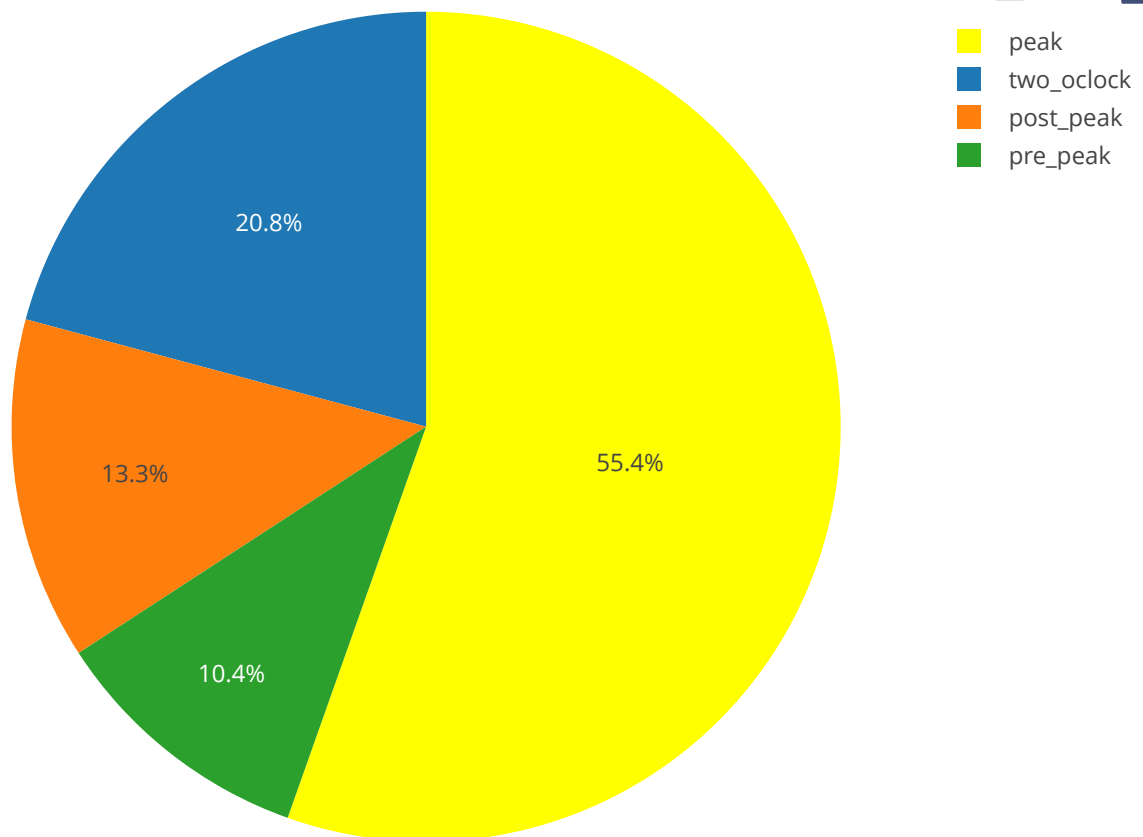
```
iday")) |>
  group_by(time_block, day) |>
  summarise(count = n(), pct_classes = count / nrow(peak_space))
```

`summarise()` has grouped output by 'time_block'. You can override using the
`.groups` argument.

```
peak_numbers = na.omit(peak_numbers)
peak_numbers$time_block = factor(peak_numbers$time_block, levels = c("pre_peak", "peak", "two_oclock", "post_peak"))
```

```
plot_ly(peak_numbers, labels = ~time_block, values = ~pct_classes, marker = list(colors = c('yellow', 'purple', 'steelblue', 'black')), type = 'pie') |> layout(title = "Overall Distribution of Classes by Time Block")
```

Overall Distribution of Classes by Time Block

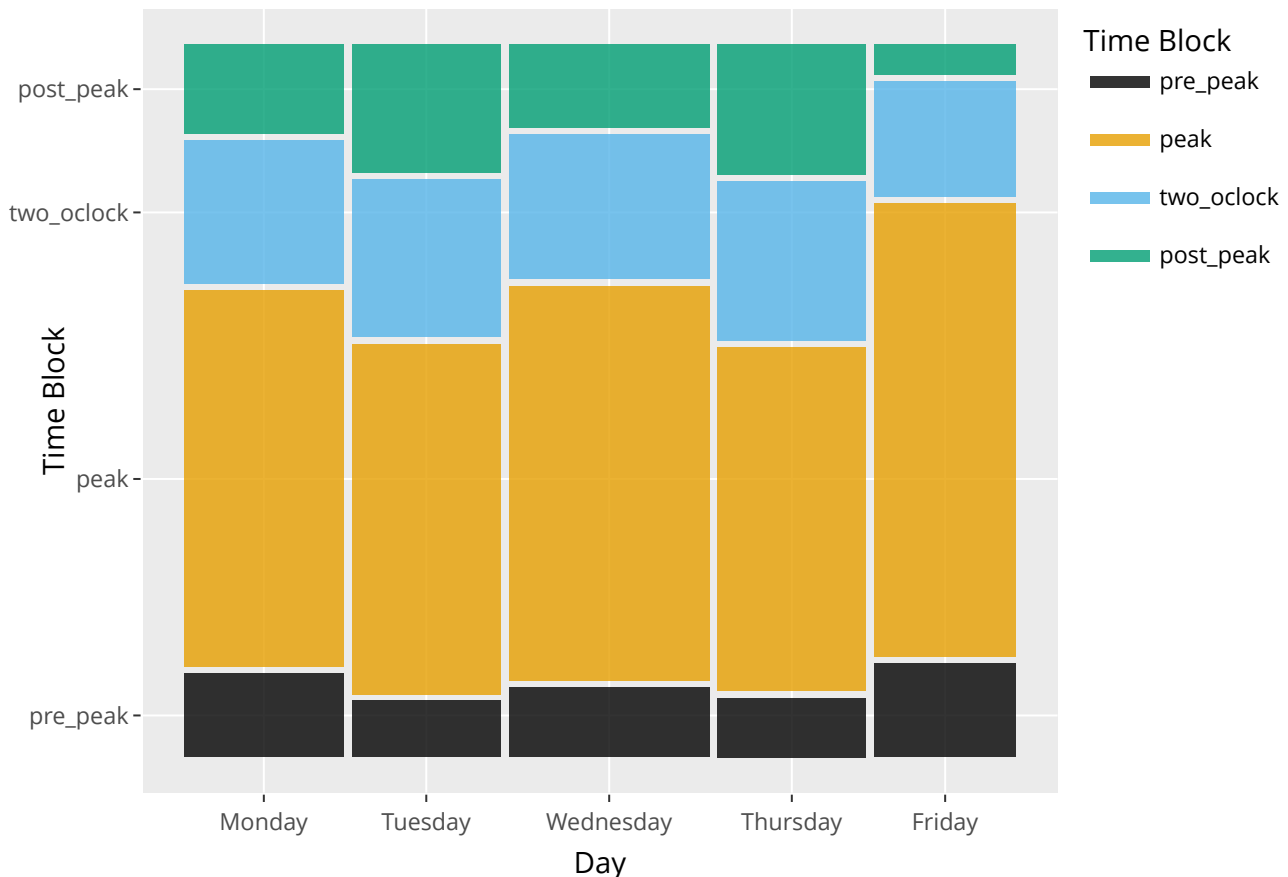


Mosaic Plot 1

```
plotly::ggplotly(ggplot(peak_numbers) + geom_mosaic(aes(weight = pct_classes, x = product(day), fill = time_block)) + labs(x = "Day", y = "Time Block", title = "Percentage of Classes by Day by Time Block", fill = "Time Block") + scale_fill_manual(values = colorblind_pal()(8)))
```

```
## Warning: `unite_()` was deprecated in tidyr 1.2.0.
## i Please use `unite()` instead.
## i The deprecated feature was likely used in the ggmosaic package.
## Please report the issue at <https://github.com/haleyjeppson/ggmosaic>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

Percentage of Classes by Day by Time Block

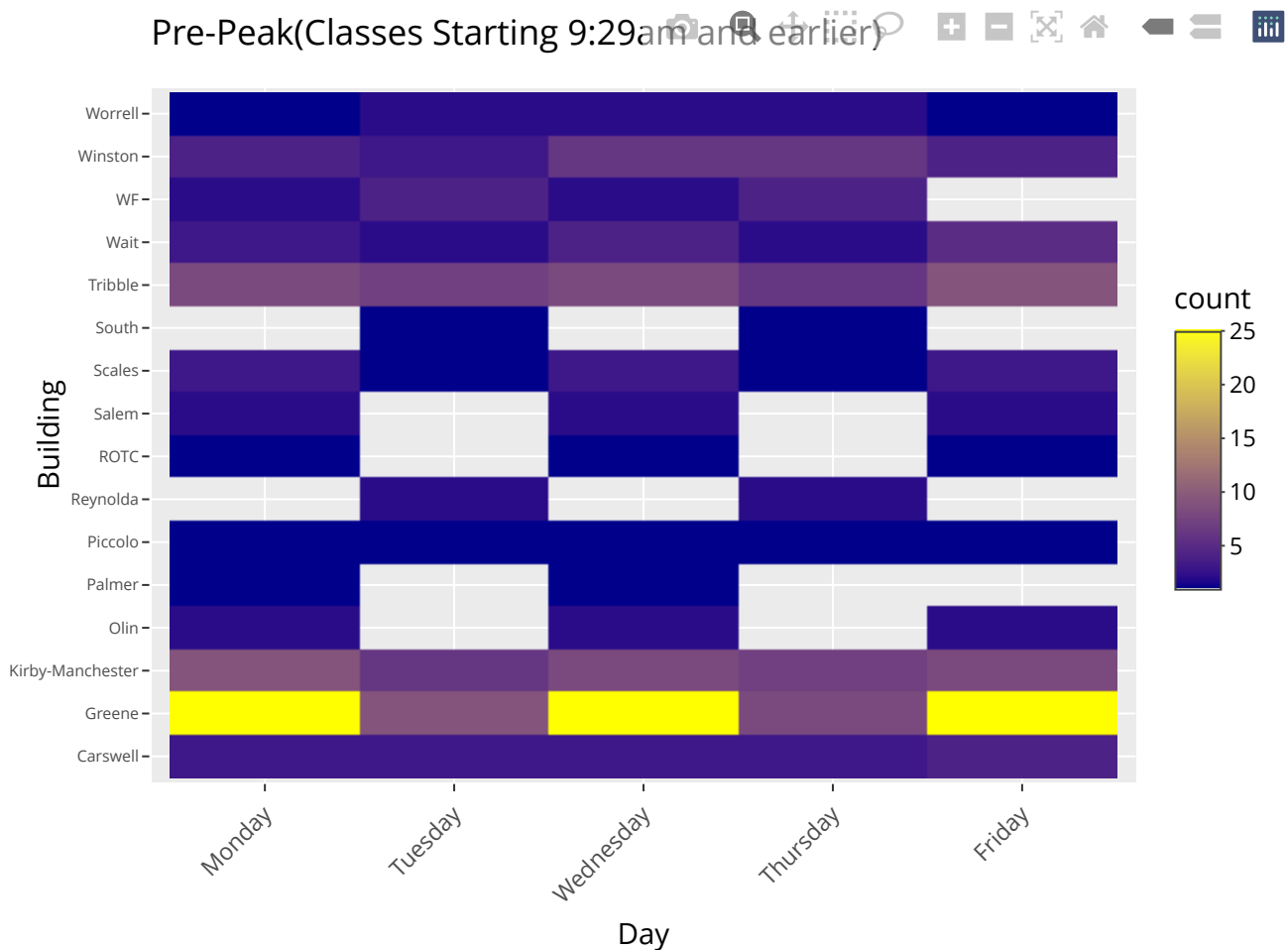


```
peak_space = meetings_long |>
  mutate(building = word(Locations__Building_Room_,1)) |> ##need to read until the first
  # space(just to adjust Kirby-Manchester)
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday"))) |>
  mutate(time_str = str_extract(Meeting_Patterns, regex("(?<=\\|)[^-]+")), time = as_hms(
    parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:
    29:00") ~ "9:30 and Before", time <= as_hms("13:59:59") ~ "9:30 to 1:59", time <= as_hms(
    "14:00:00") ~ "2:00 to 2:59", time >= as_hms("15:00:00") ~ "3:00 and later", TRUE ~ N
    A_character_))
post_peak = peak_space |>
  filter(time_block == "3:00 and later") |>
  group_by(building, day) |>
  summarise(count = n(), .groups = "drop")
peak = peak_space |>
  filter(time_block == "9:30 to 1:59") |>
  group_by(building, day) |>
  summarise(count = n(), .groups = "drop")
```

```
pre_peak = peak_space |>
  filter(time_block == "9:30 and Before") |>
  group_by(building, day) |>
  summarise(count = n(), .groups = "drop")
two_oclock = peak_space |>
  filter(time_block == "2:00 to 2:59") |>
  group_by(building, day) |>
  summarise(count = n(), .groups = "drop")
```

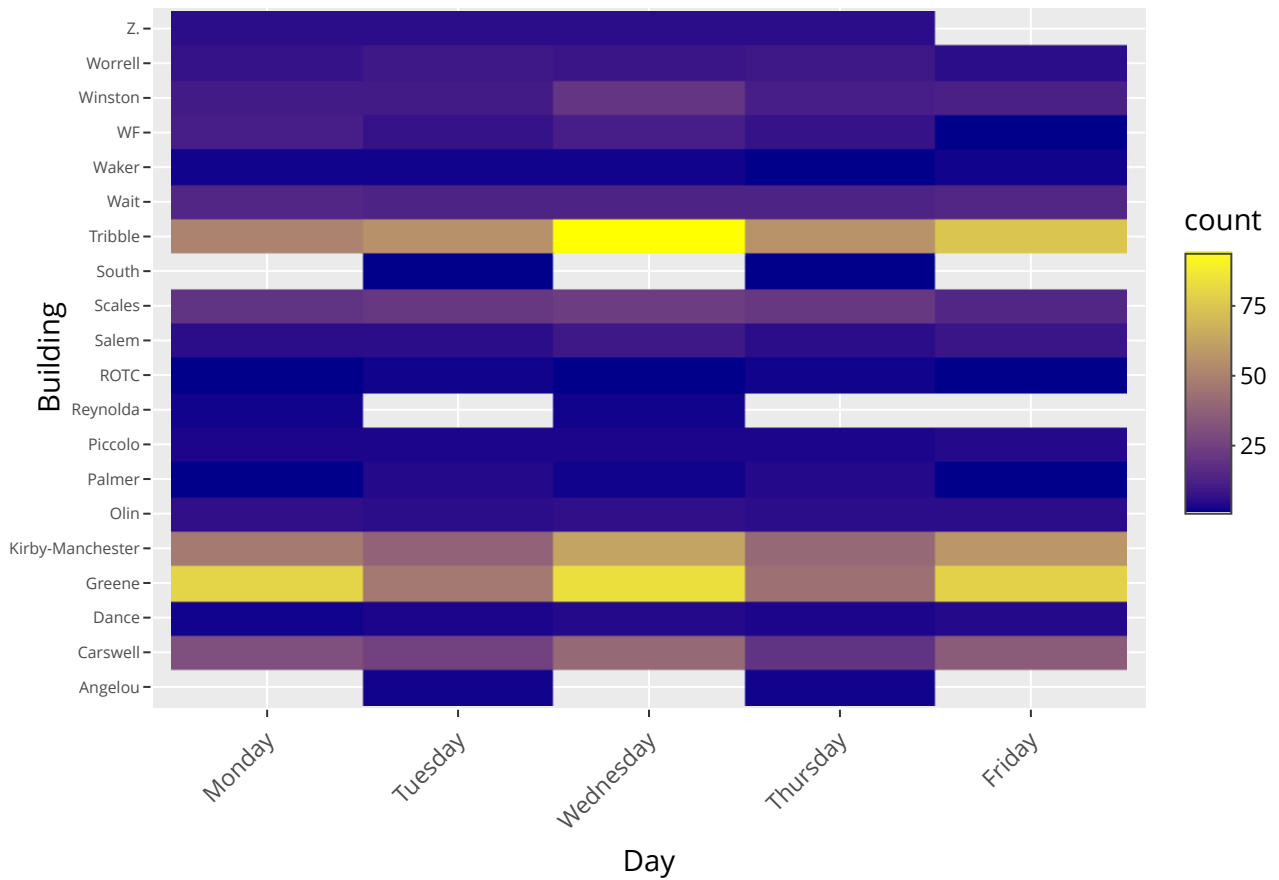
Heatmaps

```
plotly::ggplotly(ggplot(pre_peak, aes(x = day, y = building, fill = count)) + geom_tile(
  color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs(title =
  "Pre-Peak(Classes Starting 9:29am and earlier)", x = "Day", y = "Building") + theme(
  axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust =
  1)))
```



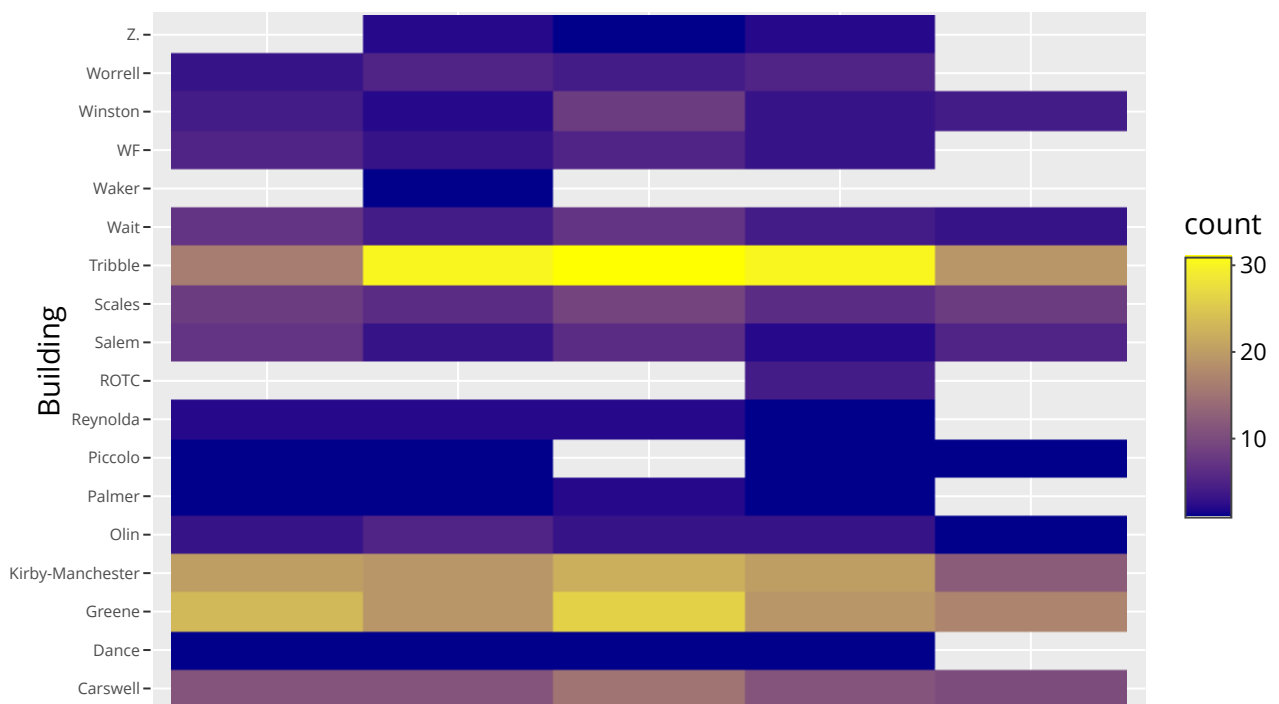
```
plotly::ggplotly(ggplot(peak, aes(x = day, y = building, fill = count)) + geom_tile(
  color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs(title =
  "Peak Time(Classes Starting between 9:30am and 1:59pm)", x = "Day", y = "Building") +
  theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust =
  1)))
```

Peak Time(Classes Starting between 9:30am and 1:59pm)



```
plotly::ggplotly(ggplot(two_oclock, aes(x = day, y = building, fill = count)) + geom_tile(color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs(title = "Peakest Peak Time(Classes Starting between 2:00 and 2:59pm)", x = "Day", y = "Building") + theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))
```

Peakest Peak Time(Classes Starting between 2:00 and 2:59pm)

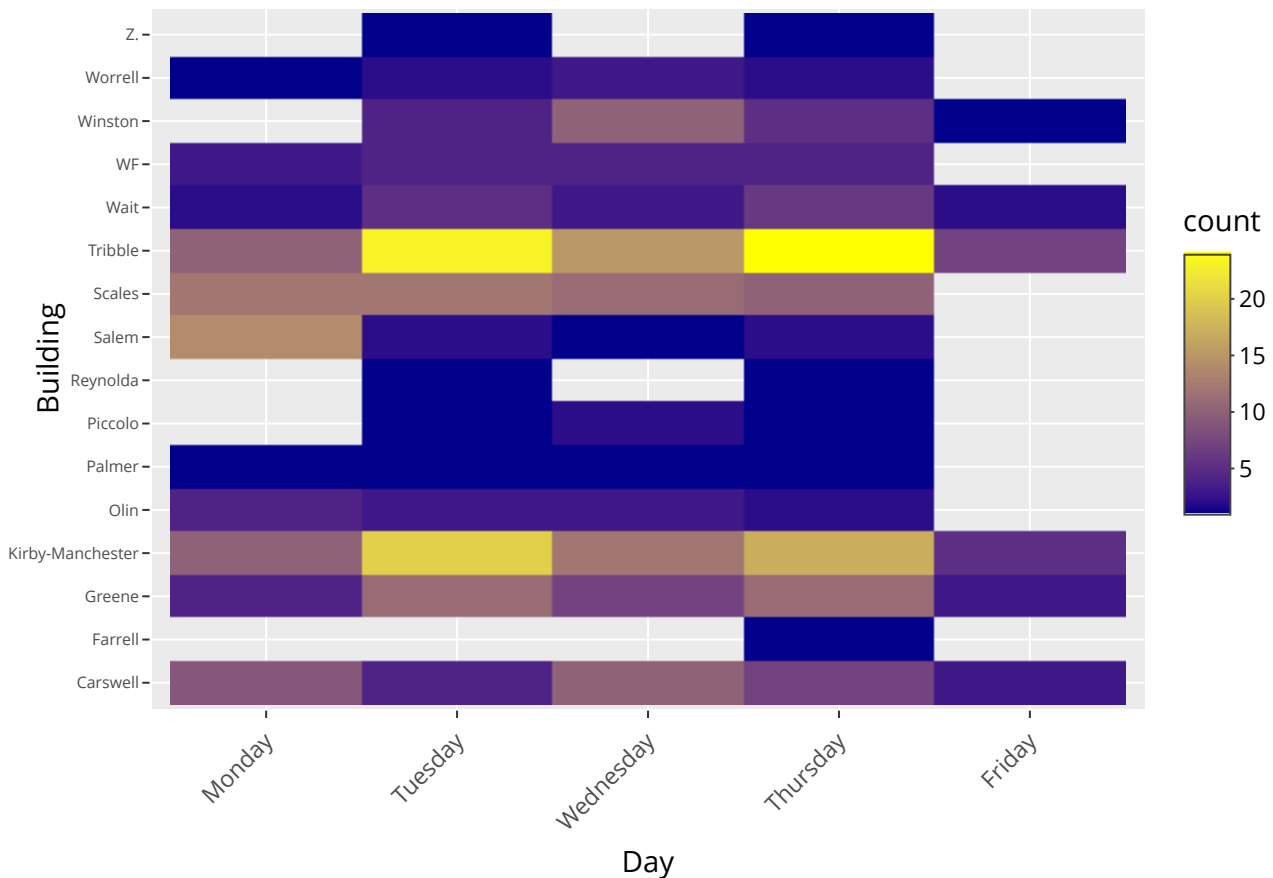


Monday Tuesday Wednesday Thursday Friday

Day

```
plotly::ggplotly(ggplot(post_peak, aes(x = day, y = building, fill = count)) + geom_tile(
  color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs(
  title = "Post-Peak(Classes Starting 3:00pm and later)", x = "Day", y = "Building") +
  theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust =
  1)))
```

Post-Peak(Classes Starting 3:00pm and later)

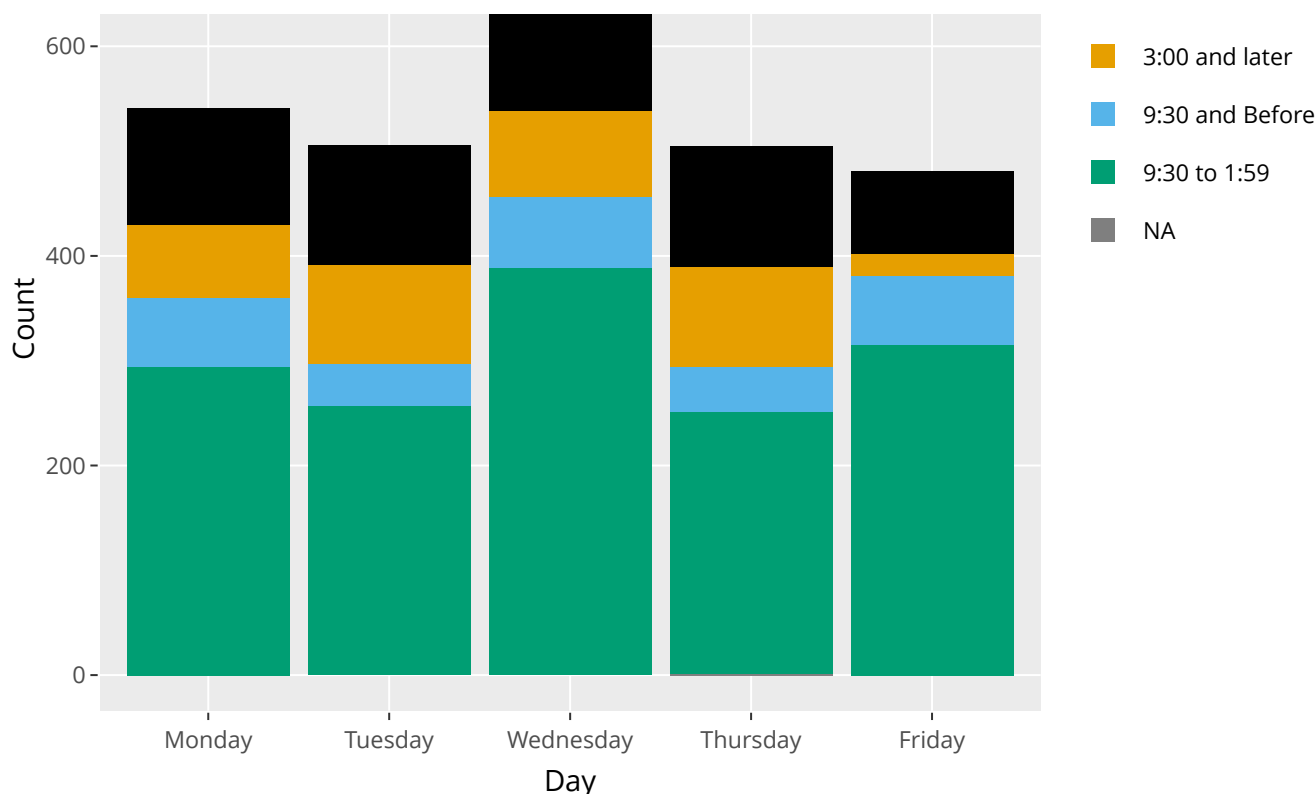


```
counts_peak = peak_space |>
  group_by(time_block, day) |>
  summarise(count = n(), .groups = "drop") |>
  arrange(desc(count))
```

```
ggplotly(ggplot(data = counts_peak, aes(x = day, y = count, fill = time_block)) + geom_bar(
  stat = "identity") + scale_fill_manual(values = colorblind_pal()(8)) + labs(x = "Day",
  y = "Count", title = "Classes per Day for Each Time Block", fill = "Time Block"))
```

Classes per Day for Each Time Block





General Use Room Work

```
gen_use_rooms = read.csv("/Users/omcginnis/Downloads/Wake Forest College Reynolda and W
ake Downtown Campus General-Use Classrooms and Student Seat Count_Updated August 2025.x
lsx - Sheet1.csv")
gen_use = gen_use_rooms |>
  select(Room) |>
  set_names("Locations__Building_Room_")
gen_use_space = inner_join(reynolda_campus, gen_use, by = "Locations__Building_Room_")
```

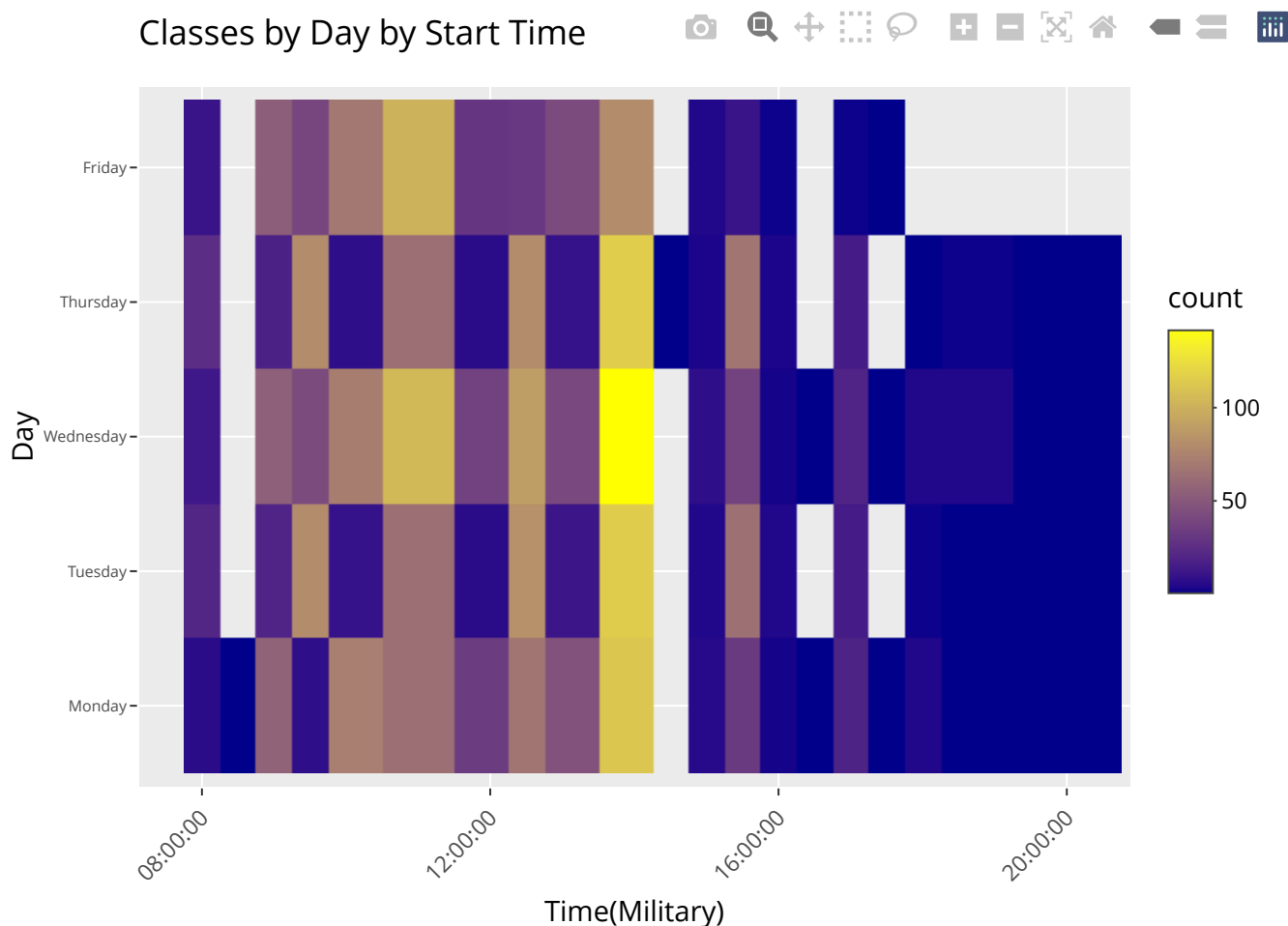
```
start_time_counts = reynolda_campus |>
mutate(time_str = str_extract(Meeting_Patterns, regex("(?<=\\|)[^~]+")), time = as_hms
(parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:
29:00") ~ "pre_peak", time <= as_hms("13:59:59") ~ "peak", time <= as_hms("14:00:00") ~
"two_oclock", time >= as_hms("15:00:00") ~ "post_peak", TRUE ~ NA_character_))|>
  mutate(days = str_extract(Meeting_Patterns, "^^[^~]+"), is_Monday = str_detect(days,
"M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thur
sday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
  filter(has_class) |>
  mutate(day = str_remove(day, "is_")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Fr
iday")) |>
  group_by(time, day) |>
  summarise(count = n()) |>
  arrange(desc(count))
```

`summarise()` has grouped output by 'time'. You can override using the

```
## `.groups` argument.
```

Classes by Day By Start Time

```
plotly::ggplotly(ggplot(start_time_counts, aes(x = time, y = day, fill = count)) + geom_tile(color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs(x = "Time(Military)", y = "Day", title = "Classes by Day by Start Time") + theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))
```



General Use Visualizations

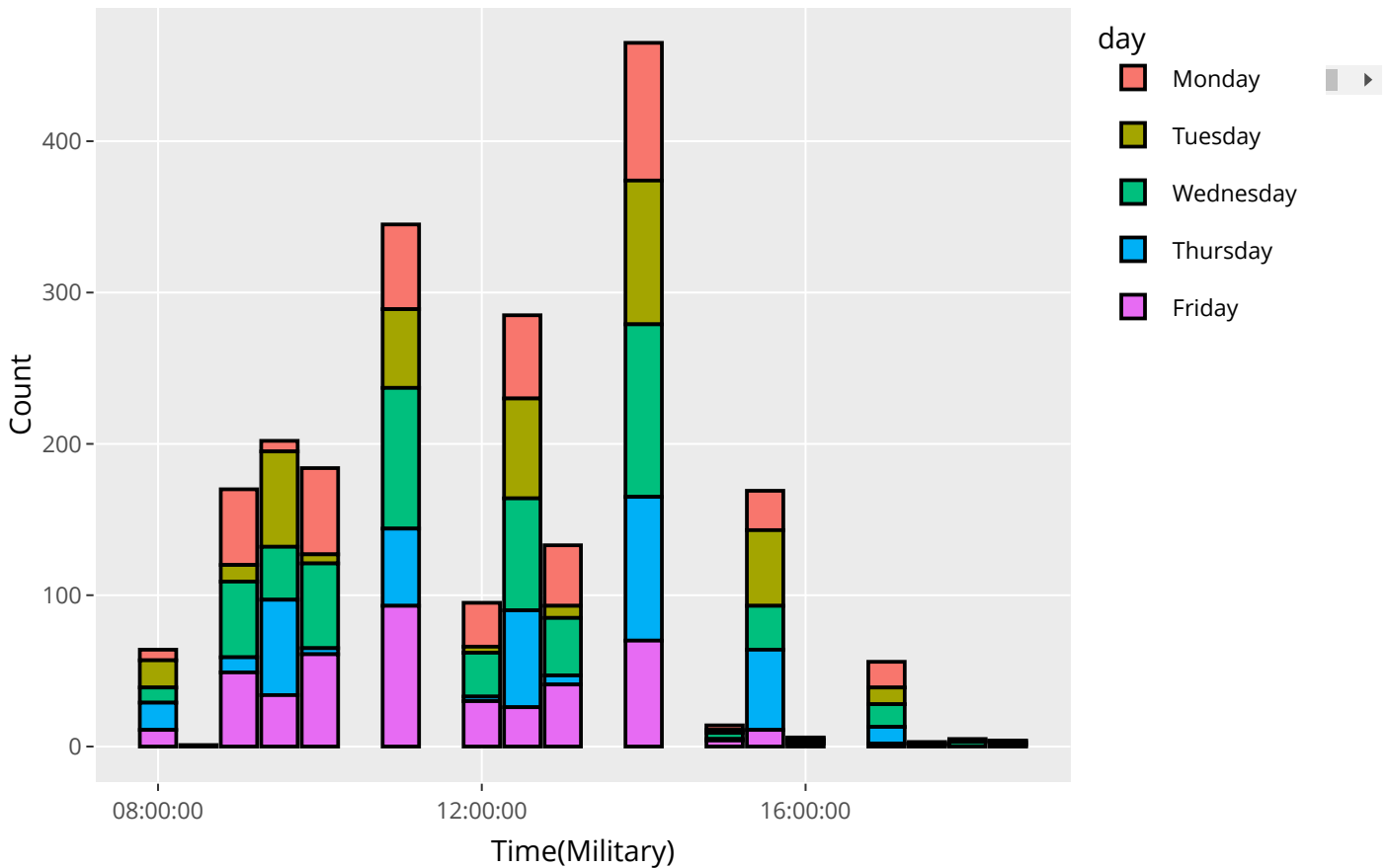
```
gen_use_counts = gen_use_space |>
mutate(time_str = str_extract(Meeting_Patterns, regex("(?<=\\|)[^-]+")), time = as_hms(
  parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:
29:00") ~ "pre_peak", time <= as_hms("13:59:59") ~ "peak", time <= as_hms("14:00:00") ~
"two_oclock", time >= as_hms("15:00:00") ~ "post_peak", TRUE ~ NA_character_))|>
mutate(days = str_extract(Meeting_Patterns, "^^[^|]+"), is_Monday = str_detect(days,
"M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thur
sday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
filter(has_class) |>
mutate(day = str_remove(day, "is_")) |>
mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Fr
iday")))|>
```

```
group_by(time, day) |>
summarise(count = n()) |>
arrange(desc(count))
```

`summarise()` has grouped output by 'time'. You can override using the
`.groups` argument.

```
plotly::ggplotly(ggplot(gen_use_counts, aes(x = time, y = count, fill = day)) + geom_bar(
  stat = "identity", color = "black") + labs(x = "Time(Military)", y = "Count", title =
  "Classes by Day in General-Use Classrooms by Start Time"))
```

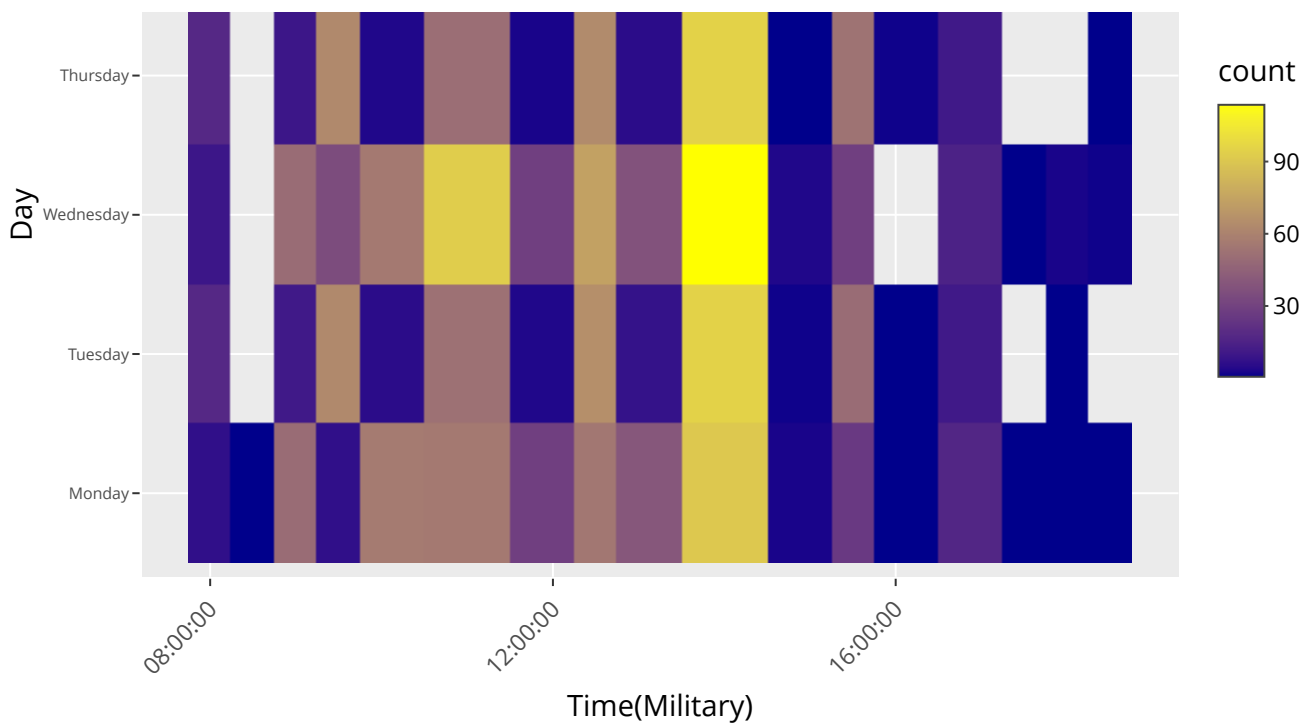
Classes by Day in General-Use Classrooms by Start Time



```
plotly::ggplotly(ggplot(gen_use_counts, aes(x = time, y = day, fill = count)) + geom_tile(
  color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs(x =
  "Time(Military)", y = "Day", title = "Classes by Day in General-Use Classrooms by Start
  Time") + theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle =
  45, hjust = 1)))
```

Classes by Day in General-Use Classrooms by Start Time





```
gen_use_counts = gen_use_space |>
  mutate(days = str_extract(Meeting_Patterns, "^[^|]+"), is_Monday = str_detect(days,
"M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thur
sday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
  filter(has_class) |>
  mutate(day = str_remove(day, "is_")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Fr
iday"))) |>
  mutate(time_str = str_extract(Meeting_Patterns, regex("(?<=\\|)[^|]+")), time = as_hms
(parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:
29:00") ~ "Pre-9:30", time <= as_hms("13:59:59") ~ "Between 9:30 and 1:59", time <= as_
hms("14:00:00") ~ "Between 2:00 and 2:59", time >= as_hms("15:00:00") ~ "3:00 and Afte
r", TRUE ~ NA_character_))|>
  group_by(time_block, day) |>
  summarise(count = n(), pct_classes = n()/ nrow(gen_use_space))
```

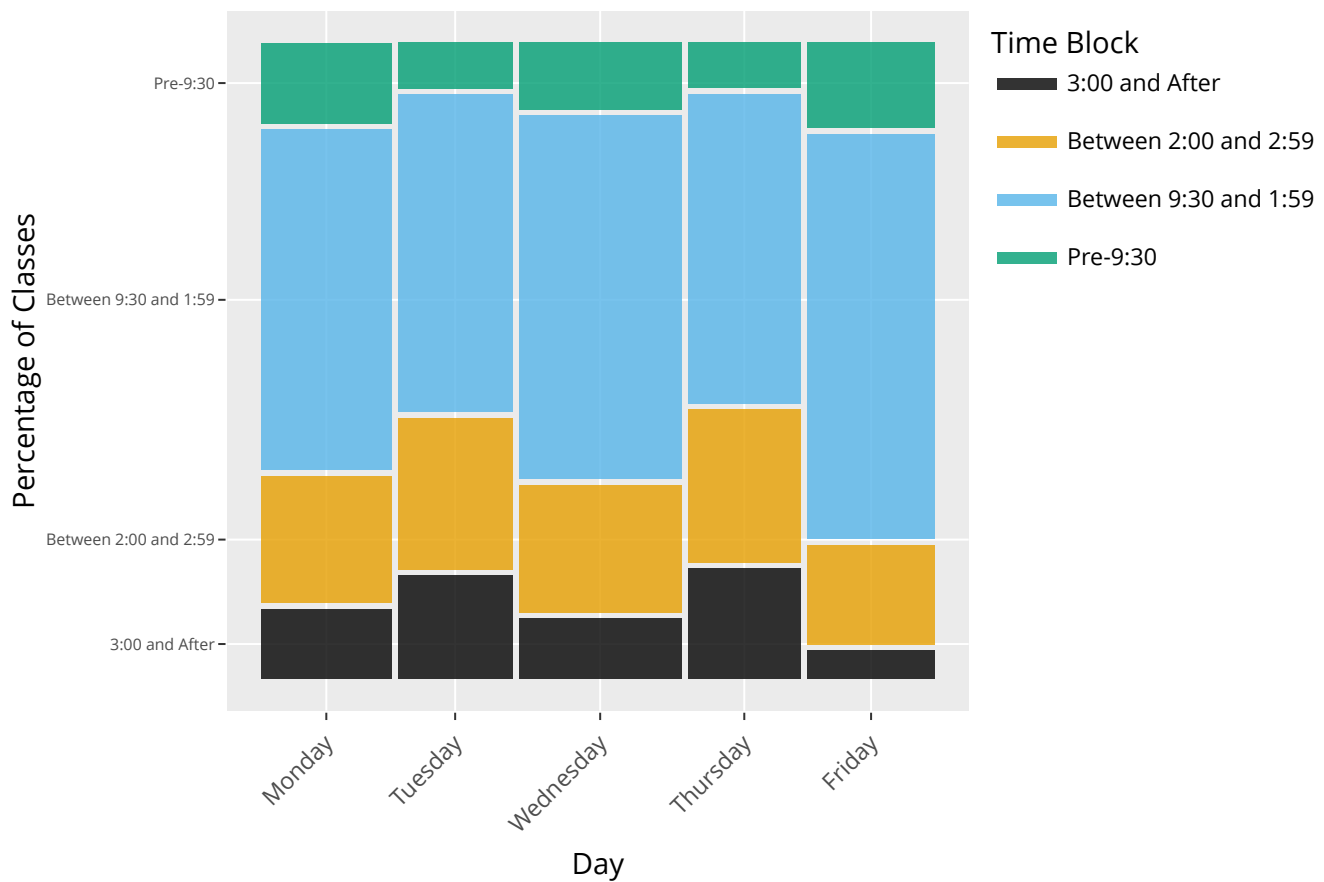
`summarise()` has grouped output by 'time_block'. You can override using the
`.groups` argument.

```
gen_use_counts = na.omit(gen_use_counts)
```

Mosaic Plot 2

```
plotly::ggplotly(ggplot(gen_use_counts) + geom_mosaic(aes(x = product(day), weight = pc
t_classes, fill = time_block)) + scale_fill_manual(values = colorblind_pal()(8)) + them
e(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust =
1)) + labs(x = "Day", y = "Percentage of Classes", title = "Percentage of Classes in Ge
neral Use Classrooms by Block", fill = "Time Block"))
```

Percentage of Classes in General Use Classrooms by Block



```
gen_use_space |>
  group_by(Locations__Building_Room_) |>
  summarise(count = n()) |>
  arrange(desc(count))
```

```
## # A tibble: 120 × 2
##   Locations__Building_Room_      count
##   <chr>                    <int>
## 1 Salem Hall 012             18
## 2 Wait Chapel - Divinity-Religion 314  15
## 3 Piccolo Hall 122            14
## 4 Carswell Hall 302           13
## 5 Greene Hall 313             13
## 6 Tribble Hall B216           13
## 7 Tribble Hall C3             13
## 8 Tribble Hall C316           13
## 9 Greene Hall 145             12
## 10 Kirby-Manchester 102        12
## # i 110 more rows
```

Downtown

```
downtown = read.csv("/Users/omcginnis/Downloads/WakeDowntown.csv")
downtown = downtown |>
```

```

filter(Room.ID != "IQ-BLG60SOUTH-3506") |>
  select(c("Academic.Period", "Department", "Campus", "Course.Number", "Section.Number", "Section", "Enrollment.Count", "Meeting.Patterns", "Meeting.Day.Patterns", "Room.ID", "Delivery.Mode"))
downtown_counts = downtown |>
  mutate(time_str = str_extract(Meeting.Patterns, regex("(?<=\\|)[^-]+")), time = as_hms(parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:29:00") ~ "pre_peak", time <= as_hms("13:59:59") ~ "peak", time <= as_hms("14:00:00") ~ "two_oclock", time >= as_hms("15:00:00") ~ "post_peak", TRUE ~ NA_character_))|>
  mutate(days = str_extract(Meeting.Patterns, "^[^|]+"), is_Monday = str_detect(days, "M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thursday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
  filter(has_class) |>
  mutate(day = str_remove(day, "is_")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday"))) |>
  group_by(time, day) |>
  summarise(count = n()) |>
  arrange(desc(count))

```

`summarise()` has grouped output by 'time'. You can override using the
`.groups` argument.

```

by_room = downtown |>
  mutate(time_str = str_extract(Meeting.Patterns, regex("(?<=\\|)[^-]+")), time = as_hms(parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:29:00") ~ "pre_peak", time <= as_hms("13:59:59") ~ "peak", time <= as_hms("14:00:00") ~ "two_oclock", time >= as_hms("15:00:00") ~ "post_peak", TRUE ~ NA_character_))|>
  mutate(days = str_extract(Meeting.Patterns, "^[^|]+"), is_Monday = str_detect(days, "M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thursday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
  filter(has_class) |>
  mutate(day = str_remove(day, "is_")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday"))) |>
  group_by(Room.ID, day) |>
  summarise(count = n()) |>
  arrange(desc(count))

```

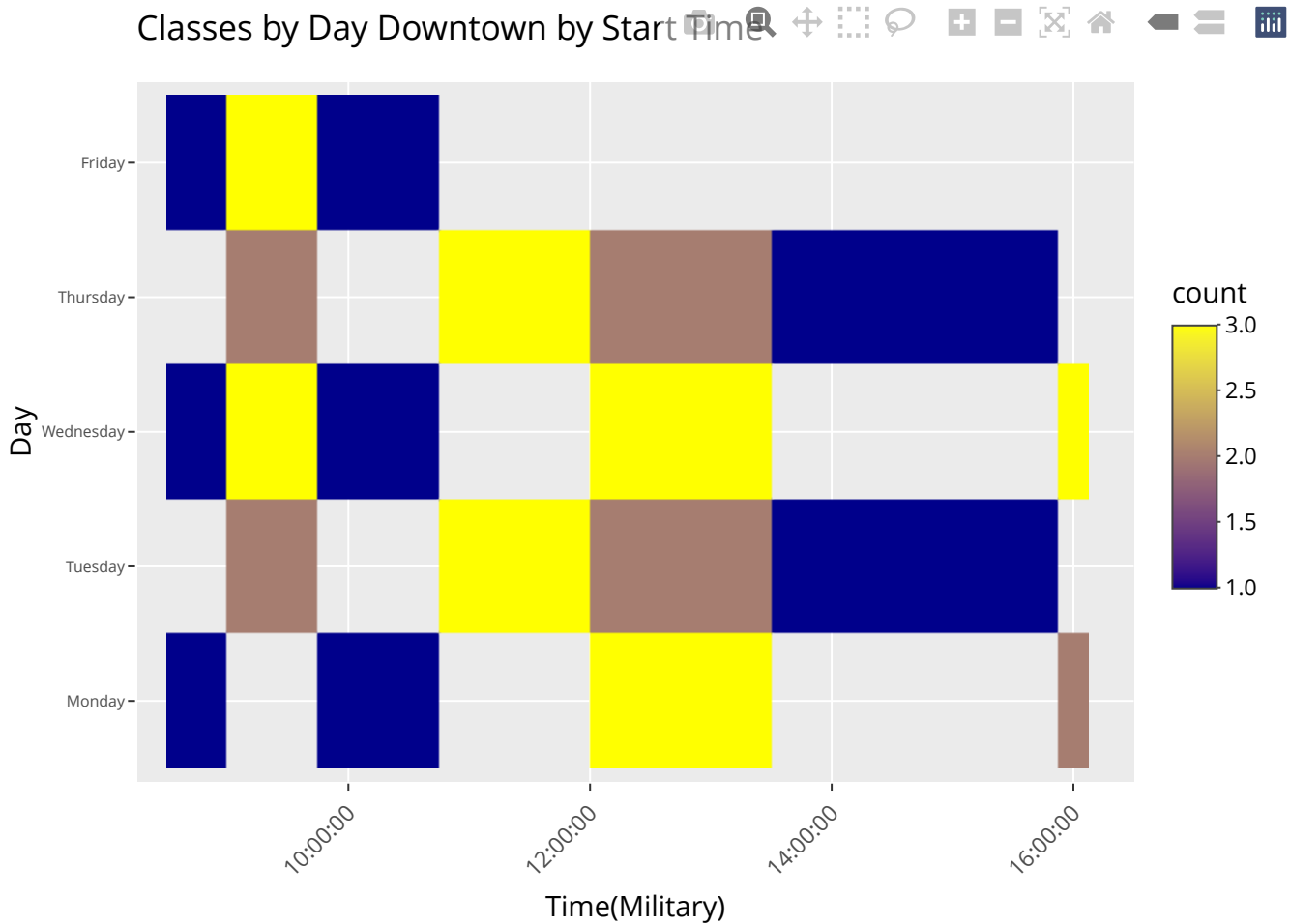
`summarise()` has grouped output by 'Room.ID'. You can override using the
`.groups` argument.

```

plotly::ggplotly(ggplot(downtown_counts, aes(x = time, y = day, fill = count)) + geom_tile(color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs(x = "Time(Military)", y = "Day", title = "Classes by Day Downtown by Start Time") + theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))

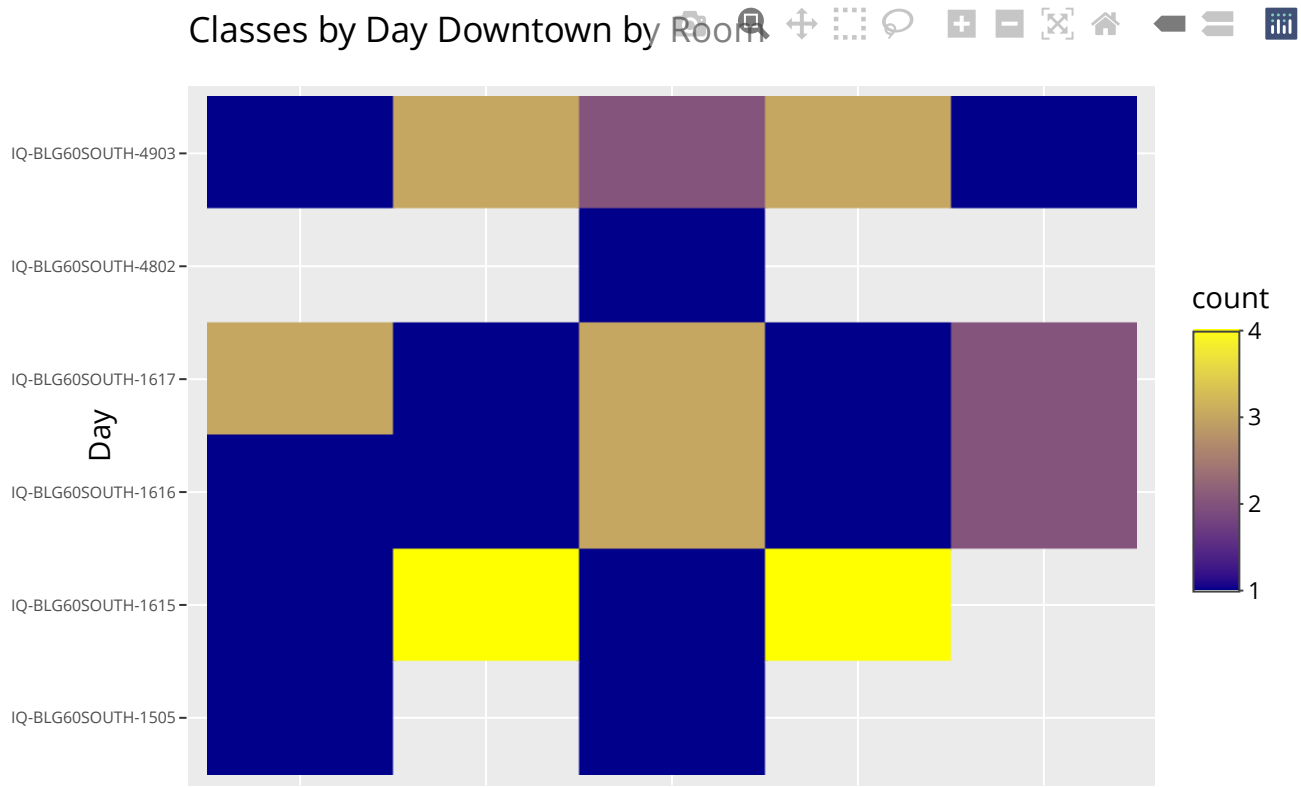
```

Classes by Day Downtown by Start Time



```
plotly::ggplotly(ggplot(by_room, aes(x = day, y = Room.ID, fill = count)) + geom_tile(
  color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs(x = "Time(Military)", y = "Day", title = "Classes by Day Downtown by Room") + theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))
```

Classes by Day Downtown by Room



Monday

Tuesday

Wednesday

Thursday

Friday

Time(Military)

Half Semester 1

```
second_half = reynolda_campus |>
  filter(Academic_Period != "2025 Fall First Half of Term")
first_half = reynolda_campus |>
  filter(Academic_Period != "2025 Fall Second Half of Term")
```

```
first_half_counts = first_half |>
  mutate(time_str = str_extract(Meeting_Patterns, regex("(?<=\\|)[^-]+")), time = as_hms(
    parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:
    29:00") ~ "pre_peak", time <= as_hms("13:59:59") ~ "peak", time <= as_hms("14:00:00") ~
    "two_oclock", time >= as_hms("15:00:00") ~ "post_peak", TRUE ~ NA_character_))|>
  mutate(days = str_extract(Meeting_Patterns, "^[^|]+"), is_Monday = str_detect(days,
    "M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thur
    sday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
  filter(has_class) |>
  mutate(day = str_remove(day, "is_")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Fr
    iday")) |>
  group_by(time, day) |>
  summarise(count = n()) |>
  arrange(desc(count))
```

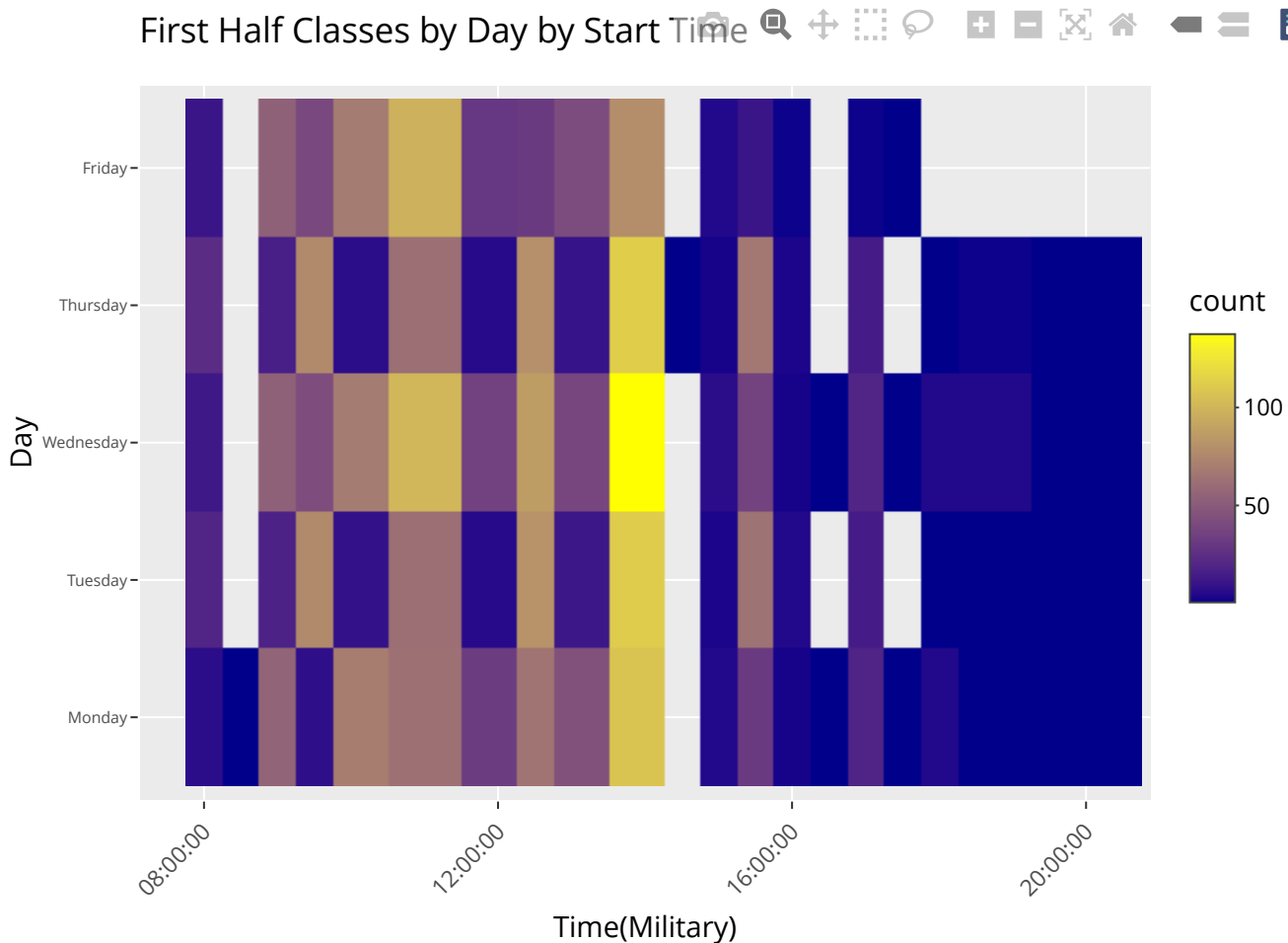
`summarise()` has grouped output by 'time'. You can override using the
`.groups` argument.

```
second_half_counts = second_half |>
  mutate(time_str = str_extract(Meeting_Patterns, regex("(?<=\\|)[^-]+")), time = as_hms(
    parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:
    29:00") ~ "pre_peak", time <= as_hms("13:59:59") ~ "peak", time <= as_hms("14:00:00") ~
    "two_oclock", time >= as_hms("15:00:00") ~ "post_peak", TRUE ~ NA_character_))|>
  mutate(days = str_extract(Meeting_Patterns, "^[^|]+"), is_Monday = str_detect(days,
    "M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thur
    sday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
  filter(has_class) |>
  mutate(day = str_remove(day, "is_")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Fr
    iday")) |>
  group_by(time, day) |>
  summarise(count = n()) |>
  arrange(desc(count))
```



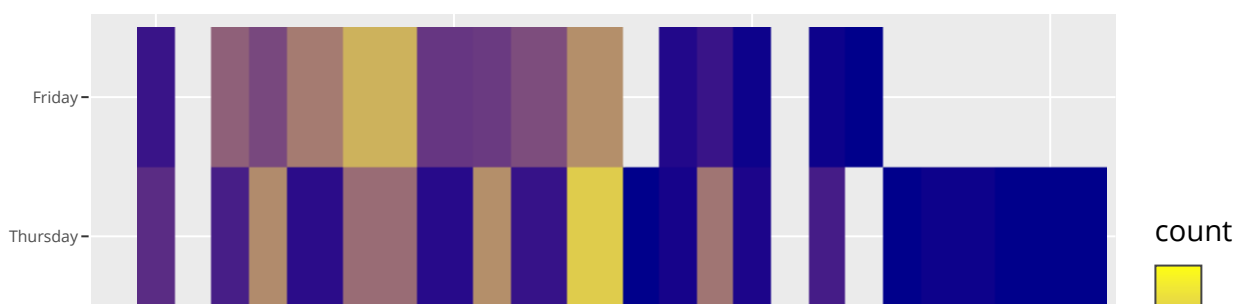
```
## `summarise()` has grouped output by 'time'. You can override using the  
## `.groups` argument.
```

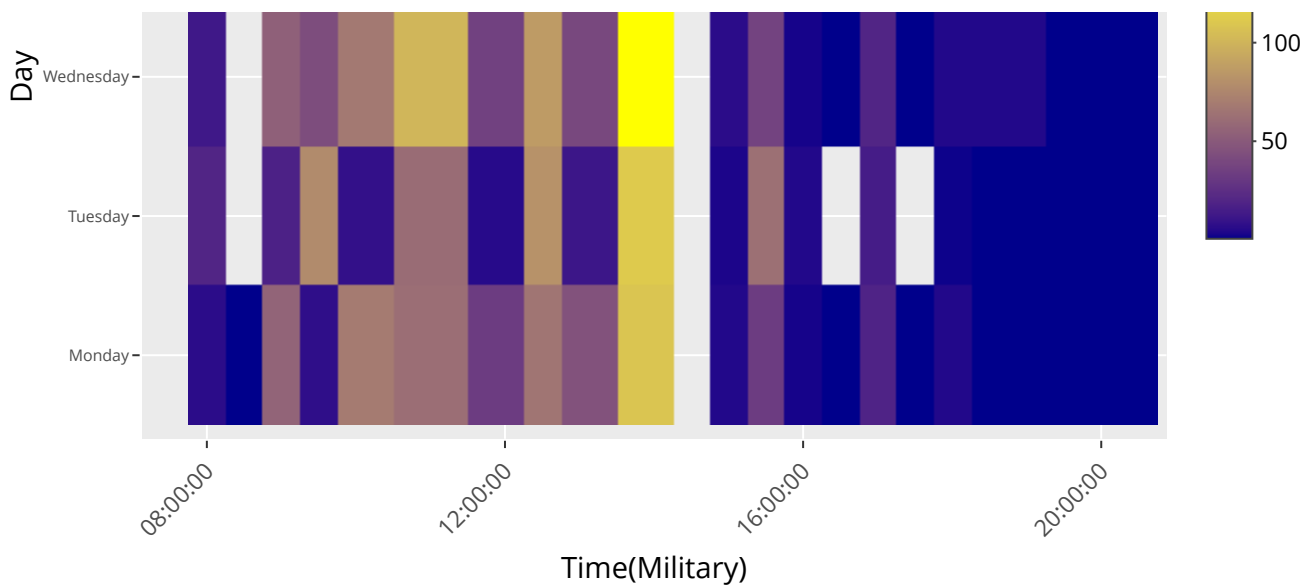
```
plotly::ggplotly(ggplot(first_half_counts, aes(x = time, y = day, fill = count)) + geom_
_tile(color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs
(x = "Time(Military)", y = "Day", title = "First Half Classes by Day by Start Time") +
theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjus
t = 1)))
```



```
plotly::ggplotly(ggplot(second_half_counts, aes(x = time, y = day, fill = count)) + geo
m_tile(color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs
(x = "Time(Military)", y = "Day", title = "Second Half Classes by Day by Start Time")
+ theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hju
st = 1)))
```

Second Half Classes by Day by Start Time





Half Semester 2

```
first_half = reynolda_campus |>
  filter(Academic_Period == "2025 Fall First Half of Term")
second_half = reynolda_campus |>
  filter(Academic_Period == "2025 Fall Second Half of Term")
```

```
first_half_counts = first_half |>
  mutate(time_str = str_extract(Meeting_Patterns, regex("(?<=\\|)[^-]+")), time = as_hms(
    parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:
    29:00") ~ "pre_peak", time <= as_hms("13:59:59") ~ "peak", time <= as_hms("14:00:00") ~
    "two_oclock", time >= as_hms("15:00:00") ~ "post_peak", TRUE ~ NA_character_))|>
  mutate(days = str_extract(Meeting_Patterns, "^[^|]+"), is_Monday = str_detect(days,
    "M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thur
    sday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
  filter(has_class) |>
  mutate(day = str_remove(day, "is_")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Fr
    iday")) |>
  group_by(time, day) |>
  summarise(count = n()) |>
  arrange(desc(count))
```

`summarise()` has grouped output by 'time'. You can override using the
`.groups` argument.

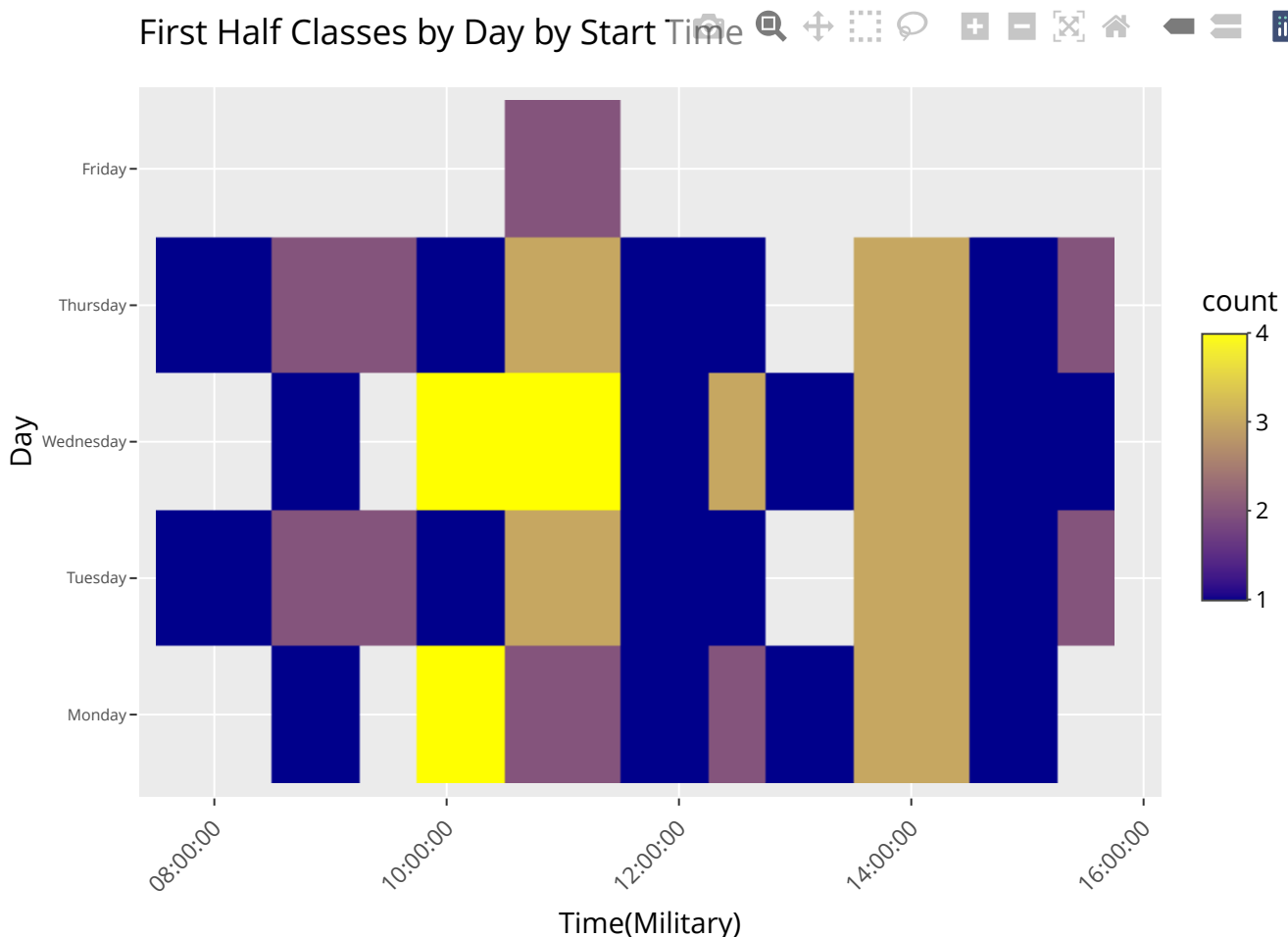
```
second_half_counts = second_half |>
  mutate(time_str = str_extract(Meeting_Patterns, regex("(?<=\\|)[^-]+")), time = as_hms(
    parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:
    29:00") ~ "pre_peak", time <= as_hms("13:59:59") ~ "peak", time <= as_hms("14:00:00") ~
    "two_oclock", time >= as_hms("15:00:00") ~ "post_peak", TRUE ~ NA_character_))|>
  mutate(days = str_extract(Meeting_Patterns, "^[^|]+"), is_Monday = str_detect(days,
    "M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thur
```

```
sday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
  filter(has_class) |>
  mutate(day = str_remove(day, "is_")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday"))) |>
  group_by(time, day) |>
  summarise(count = n()) |>
  arrange(desc(count))
```

`summarise()` has grouped output by 'time'. You can override using the
`.groups` argument.

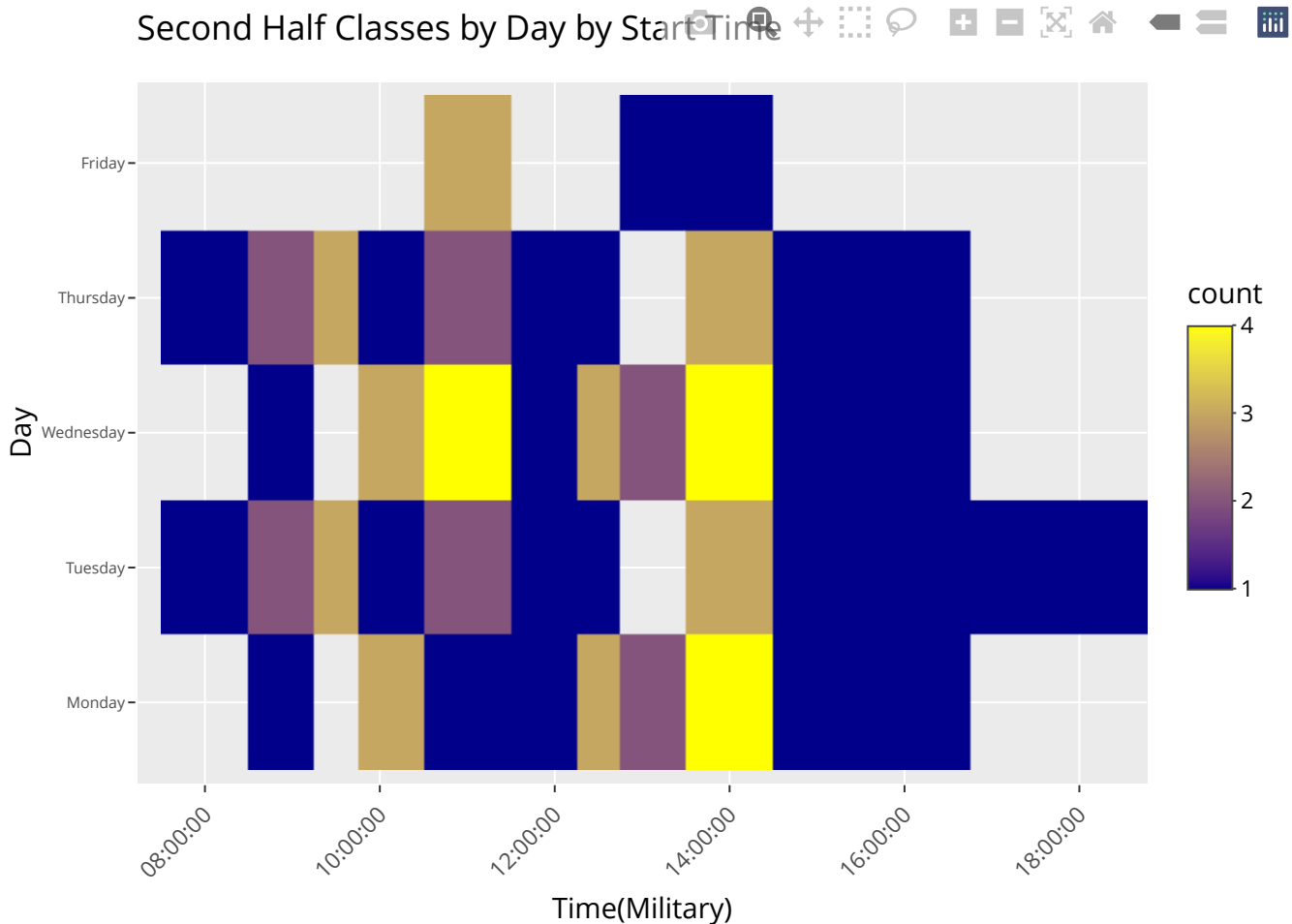
```
plotly::ggplotly(ggplot(first_half_counts, aes(x = time, y = day, fill = count)) + geom_
_tile(color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs
(x = "Time(Military)", y = "Day", title = "First Half Classes by Day by Start Time") +
theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))
```

First Half Classes by Day by Start Time



```
plotly::ggplotly(ggplot(second_half_counts, aes(x = time, y = day, fill = count)) + geom_
_tile(color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs
(x = "Time(Military)", y = "Day", title = "Second Half Classes by Day by Start Time")
+ theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))
```

Second Half Classes by Day by Start Time



```
first_buildings = first_half |>
  mutate(building = str_extract(Locations__Building_Room_, "^[A-Za-z]+")) |>
  mutate(building = word(Locations__Building_Room_,1)) |> ##need to read until the first space(just to adjust Kirby-Manchester)
  group_by(building, Meeting_Patterns) |>
  mutate(days = str_extract(Meeting_Patterns, "^[^|]+"), is_Monday = str_detect(days, "M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thursday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
  filter(has_class) |>
  mutate(day = str_remove(day, "is_")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday"))) |>
  mutate(time_str = str_extract(Meeting_Patterns, regex("(?<=\\|)[^|]+")), time = as_hms(parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:29:00") ~ "9:30 and Before", time <= as_hms("13:59:59") ~ "9:30 to 1:59", time <= as_hms("14:00:00") ~ "2:00 to 2:59", time >= as_hms("15:00:00") ~ "3:00 and later", TRUE ~ NA_character_)) |>
  group_by(building, day) |>
  summarise(count = n(), .groups = "drop")
second_buildings = second_half |>
  mutate(building = str_extract(Locations__Building_Room_, "^[A-Za-z]+")) |>
  mutate(building = word(Locations__Building_Room_,1)) |> ##need to read until the first space(just to adjust Kirby-Manchester)
  group_by(building, Meeting_Patterns) |>
  mutate(days = str_extract(Meeting_Patterns, "^[^|]+"), is_Monday = str_detect(days,
```

```

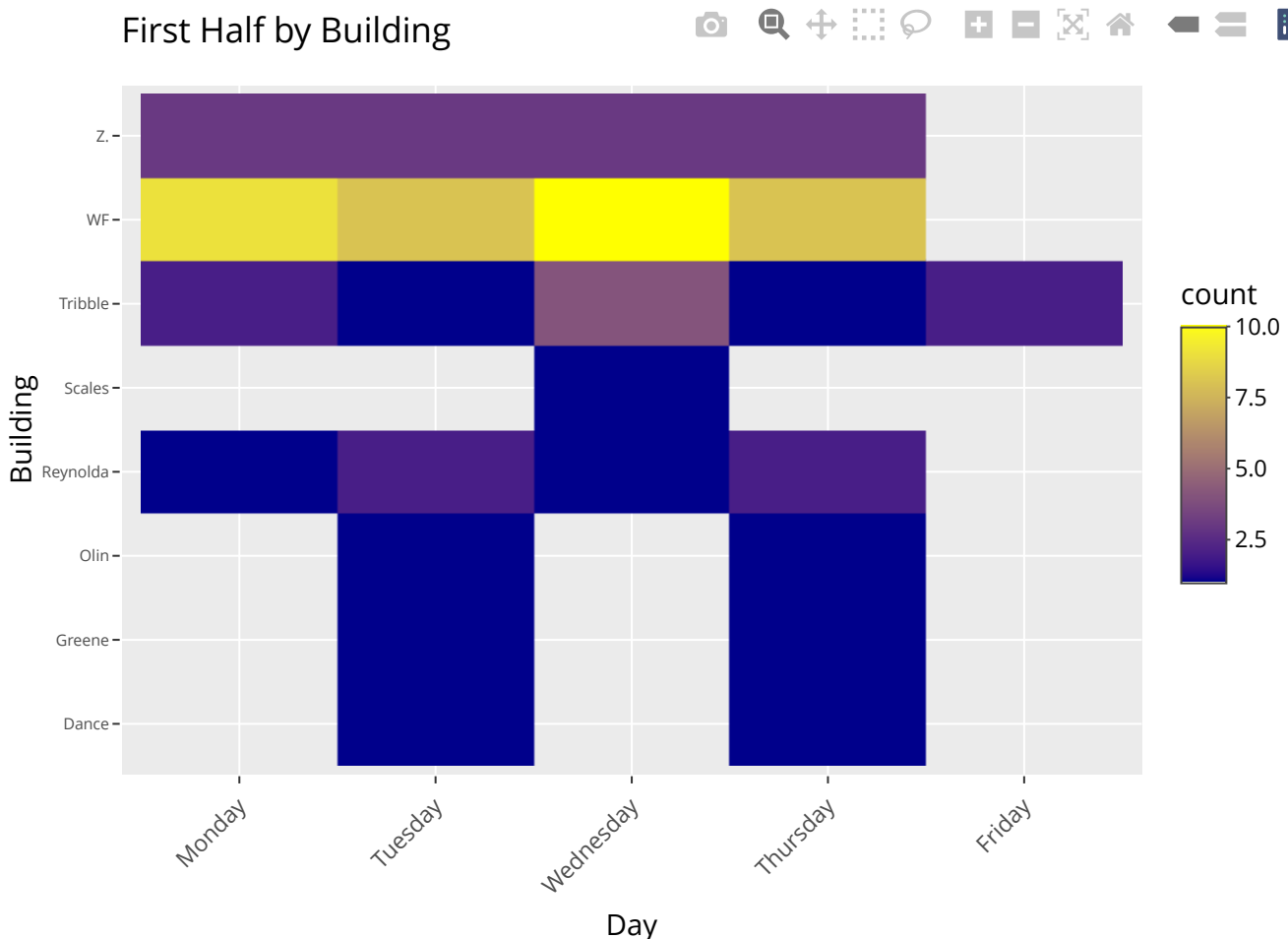
"M"), is_Tuesday = str_detect(days, "T"), is_Wednesday = str_detect(days, "W"), is_Thursday = str_detect(days, "R"), is_Friday = str_detect(days, "F")) |>
  pivot_longer(cols = starts_with("is_"), names_to = "day", values_to = "has_class") |>
  filter(has_class) |>
  mutate(day = str_remove(day, "is_")) |>
  mutate(day = factor(day, levels = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday"))) |>
  mutate(time_str = str_extract(Meeting_Patterns, regex("(?<=\\|)[^-]+")), time = as_hms(
    parse_date_time(time_str, orders = "I:Mp")), time_block = case_when(time < as_hms("09:29:00") ~ "9:30 and Before", time <= as_hms("13:59:59") ~ "9:30 to 1:59", time <= as_hms("14:00:00") ~ "2:00 to 2:59", time >= as_hms("15:00:00") ~ "3:00 and later", TRUE ~ NA_character_)) |>
  group_by(building, day) |>
  summarise(count = n(), .groups = "drop")

```

```

plotly::ggplotly(ggplot(first_buildings, aes(x = day, y = building, fill = count)) + geom_tile(color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs(title = "First Half by Building", x = "Day", y = "Building") + theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))

```



```

plotly::ggplotly(ggplot(second_buildings, aes(x = day, y = building, fill = count)) + geom_tile(color = "black") + scale_fill_gradient(low = "darkblue", high = "yellow") + labs(title = "Second Half by Building", x = "Day", y = "Building") + theme(axis.text.y = element_text(size = 6), axis.text.x = element_text(angle = 45, hjust = 1)))

```

Second Half by Building

